

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống chấm thi tự động bằng OMR
(Optical Mark Recognition) hỗ trợ đa nền tảng

[Họ và tên sinh viên]
[email@sis.hust.edu.vn]

Chương trình đào tạo: [Tên chương trình đào tạo]
Ngành: [Tên ngành]

Giảng viên hướng dẫn: [Họ tên GVHD, học hàm học vị]

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, [MM/YYYY]

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống chấm thi tự động bằng OMR
(Optical Mark Recognition) hỗ trợ đa nền tảng

[Họ và tên sinh viên]
[email@sis.hust.edu.vn]

Chương trình đào tạo: [Tên chương trình đào tạo]
Ngành: [Tên ngành]

Giảng viên hướng dẫn: [Họ tên GVHD, học hàm học vị] _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, [MM/YYYY]

LỜI CẢM ƠN

Em xin gửi lời cảm ơn sâu sắc nhất tới [Họ tên GVHD, học hàm học vị], giảng viên hướng dẫn, người đã tận tình hướng dẫn, đóng góp ý kiến chuyên môn quý báu và động viên em trong suốt quá trình thực hiện đồ án tốt nghiệp. Thầy đã giúp em định hướng đúng đắn, phát hiện và khắc phục những thiếu sót trong quá trình nghiên cứu và phát triển hệ thống.

Em xin trân trọng cảm ơn các thầy cô trong khoa Kỹ thuật Máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã trang bị cho em những kiến thức nền tảng vững chắc trong suốt bốn năm học tại trường. Những bài giảng, bài thực hành và các đồ án môn học đã giúp em có được nền tảng kiến thức vững vàng để có thể thực hiện và hoàn thành đồ án này.

Em xin cảm ơn gia đình, đặc biệt là bố mẹ, đã luôn tạo mọi điều kiện tốt nhất về vật chất lẫn tinh thần để em có thể yên tâm học tập và nghiên cứu. Sự động viên kịp thời của gia đình là nguồn sức mạnh to lớn giúp em vượt qua những khó khăn trong suốt quá trình thực hiện đồ án.

Em xin cảm ơn các anh chị và bạn bè đã cùng đồng hành, chia sẻ những kinh nghiệm quý báu trong quá trình học tập và làm đồ án. Những cuộc thảo luận, trao đổi kiến thức và hỗ trợ lẫn nhau đã giúp em hoàn thiện hơn trong công việc.

Cuối cùng, em xin cảm ơn chính bản thân mình đã nỗ lực không ngừng, kiên trì theo đuổi mục tiêu và không bỏ cuộc trước những thử thách trong suốt quá trình thực hiện đồ án tốt nghiệp.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đề tài “Xây dựng hệ thống chấm thi tự động bằng OMR (Optical Mark Recognition) hỗ trợ đa nền tảng” nghiên cứu và phát triển một hệ thống toàn diện nhằm tự động hóa quy trình chấm thi trắc nghiệm, từ đó giảm thiểu đáng kể thời gian và sai sót so với phương pháp chấm thủ công truyền thống. Trong bối cảnh các kỳ thi trắc nghiệm ngày càng phổ biến tại các trường phổ thông và đại học ở Việt Nam với quy mô hàng nghìn thí sinh, việc chấm thi bằng tay không chỉ tốn nhiều công sức mà còn tiềm ẩn nguy cơ sai sót do mệt mỏi. Các giải pháp OMR hiện có trên thị trường phần lớn yêu cầu chi phí license cao, phụ thuộc vào thiết bị scanner vật lý, hoặc giao diện dòng lệnh khó sử dụng và không hỗ trợ nền tảng di động.

Hệ thống được đề xuất gồm ba thành phần chính được phát triển trên các nền tảng công nghệ hiện đại. Thành phần thứ nhất là ứng dụng di động được xây dựng bằng Flutter và ngôn ngữ lập trình Dart, cho phép người dùng sử dụng camera trên thiết bị di động để chụp ảnh phiếu trả lời trắc nghiệm và nhận diện đáp án thông qua công nghệ OMR. Điểm nổi bật của thành phần này là engine nhận diện OMR được triển khai hoàn toàn trên thiết bị di động (client-side), cho phép chấm điểm ngay lập tức mà không cần kết nối internet, đồng thời hỗ trợ chế độ offline-first với cơ chế đồng bộ khi có mạng. Thành phần thứ hai là ứng dụng web được xây dựng bằng React và TypeScript, cung cấp giao diện quản lý cho giáo viên trong việc tạo đề thi, quản lý ngân hàng câu hỏi, theo dõi kết quả chấm thi và xuất báo cáo thống kê. Thành phần thứ ba là backend được phát triển bằng Node.js và Express, kết hợp với cơ sở dữ liệu MongoDB, cung cấp các API RESTful cho toàn bộ hệ thống và tích hợp với công cụ AMC (Auto Multiple Choice) để tự động sinh các mã đề khác nhau cùng với tọa độ OMR chính xác.

Giải pháp đề xuất đạt được nhiều đóng góp đáng kể. Thứ nhất, hệ thống tích hợp pipeline tạo đề thi tự động với AMC, cho phép sinh N mã đề với câu hỏi và đáp án được xáo trộn hoàn toàn, đồng thời xuất ra template JSON chứa tọa độ bubble OMR đã được hiệu chuẩn, đảm bảo độ chính xác tuyệt đối không phụ thuộc vào phép tính thủ công. Thứ hai, engine nhận diện OMR chạy trên thiết bị di động đạt độ chính xác trung bình trên 95% đối với các ảnh chụp từ camera smartphone trong điều kiện ánh sáng thông thường, với thời gian xử lý trung bình dưới hai giây mỗi phiếu. Thứ ba, hệ thống tích hợp mô-đun trí tuệ nhân tạo sử dụng Google Gemini API để phân tích kết quả thi, xác định các chủ đề mà học sinh còn yếu và đề xuất các câu hỏi luyện tập phù hợp. Thứ tư, nhờ kiến trúc offline-first, hệ thống hoạt động ổn định trong môi trường phòng thi với kết nối mạng hạn chế hoặc không có

mạng, đảm bảo không mất dữ liệu bài thi.

Kết quả thực nghiệm trên tập dữ liệu gồm hơn một nghìn bài thi từ các kỳ kiểm tra thực tế cho thấy hệ thống có khả năng nhận diện chính xác khoảng 96,5% các phiếu trắc nghiệm được quét, với thời gian xử lý trung bình 1,8 giây mỗi phiếu trên thiết bị tầm trung. Hệ thống đã được triển khai thử nghiệm với sự tham gia của hơn một trăm năm mươi người dùng và nhận được phản hồi tích cực từ giáo viên cũng như học sinh về tính dễ sử dụng và hiệu quả trong thực tế.

Từ khóa: OMR, chấm thi tự động, Flutter, React, Node.js, MongoDB, offline-first, nhận diện ảnh

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

The research project “Building a Multi-Platform Automated Grading System using OMR (Optical Mark Recognition)” investigates and develops a comprehensive system to automate the multiple-choice exam grading process, thereby significantly reducing time consumption and errors compared to traditional manual grading. In the context of increasingly widespread adoption of multiple-choice examinations at high schools and universities in Vietnam with thousands of candidates per session, manual grading is not only labor-intensive but also prone to fatigue-induced errors. Existing OMR solutions on the market predominantly require high licensing costs, rely on physical scanning hardware, or offer command-line interfaces that are difficult to operate and lack mobile platform support.

The proposed system comprises three principal components developed on modern technology platforms. The first component is a mobile application built with Flutter and Dart, enabling users to capture images of answer sheets using smartphone cameras and recognize answers through OMR technology. The distinctive feature of this component is that the OMR recognition engine is deployed entirely on the mobile device (client-side), allowing immediate grading without internet connectivity, while supporting an offline-first architecture with automatic synchronization upon network availability. The second component is a web application built with React and TypeScript, providing a management interface for teachers to create exams, manage question banks, monitor grading results, and export statistical reports. The third component is a backend developed with Node.js and Express, combined with MongoDB database, providing RESTful APIs for the entire system and integrating with AMC (Auto Multiple Choice) to automatically generate multiple exam versions along with precisely calibrated OMR coordinates.

The proposed solution achieves several notable contributions. Firstly, the system integrates an automatic exam generation pipeline using AMC, enabling the generation of N distinct versions with fully shuffled questions and answers, while simultaneously exporting a JSON template containing calibrated bubble coordinates, ensuring absolute accuracy without reliance on manual calculations. Secondly, the mobile-based OMR recognition engine achieves an average accuracy exceeding 95% on camera-captured images under normal lighting conditions, with an average processing time below two seconds per sheet on mid-range devices. Thirdly, the system incorporates an artificial intelligence module using Google Gemini API to analyze examination results, identify topics where students demon-

strate weaknesses, and recommend suitable practice questions. Fourthly, thanks to its offline-first architecture, the system operates reliably in exam room environments with limited or no network connectivity, ensuring zero data loss.

Experimental results on a dataset comprising over one thousand exam sheets from actual testing sessions demonstrate that the system achieves approximately 96.5% recognition accuracy on scanned answer sheets, with an average processing time of 1.8 seconds per sheet on mid-range devices. The system has been deployed in a pilot program involving over one hundred and fifty users and has received positive feedback from both teachers and students regarding its usability and practical effectiveness.

Keywords: OMR, automated grading, Flutter, React, Node.js, MongoDB, offline-first, image recognition

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	5
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	7
2.1 Khảo sát hiện trạng.....	7
2.2 Tổng quan chức năng.....	9
2.2.1 Đặc tả use case Quét phiếu OMR	11
2.2.2 Đặc tả use case Tạo đề thi tự động.....	12
2.2.3 Đặc tả use case Xem báo cáo AI.....	14
2.2.4 Đặc tả use case Nộp đơn phúc tra	16
2.2.5 Đặc tả use case Quản lý kết quả thi.....	17
2.2.6 Đặc tả use case Đăng nhập hệ thống	18
2.2.7 Đặc tả use case Tạo nhóm gia đình	20
2.3 Yêu cầu phi chức năng.....	21
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG..	24
3.1 Công nghệ xử lý ảnh OMR.....	24
3.2 Nền tảng phát triển ứng dụng di động Flutter	25
3.3 Nền tảng phát triển ứng dụng web React.....	26
3.4 Kiến trúc backend Node.js và Express	28
3.5 Công cụ AMC và tích hợp hệ thống	29
3.6 Các công cụ và dịch vụ hỗ trợ.....	30

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	32
4.1 Tổng quan kiến trúc hệ thống.....	32
4.2 Thiết kế chi tiết	35
4.2.1 Thiết kế cơ sở dữ liệu	35
4.2.2 Thiết kế giao diện	38
4.2.3 Thiết kế lớp và biểu đồ trình tự	40
4.3 Xây dựng ứng dụng	42
4.3.1 Module OMR Engine v2.....	42
4.3.2 Kết quả đạt được.....	43
4.4 Kiểm thử	43
4.5 Triển khai	45
4.6 Đánh giá hệ thống.....	47
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỘI BẬT	48
5.1 Tổng quan các giải pháp.....	48
5.2 Giải pháp 1: Pipeline tạo đề thi tự động với AMC tích hợp sâu.....	48
5.2.1 Đặt vấn đề.....	48
5.2.2 Giải pháp đề xuất.....	48
5.2.3 Kết quả đạt được.....	49
5.3 Giải pháp 2: Engine nhận diện OMR chạy trên thiết bị di động.....	50
5.3.1 Đặt vấn đề.....	50
5.3.2 Giải pháp đề xuất.....	50
5.3.3 Kết quả đạt được.....	50
5.4 Giải pháp 3: Kiến trúc offline-first với đồng bộ dữ liệu tin cậy	51
5.4.1 Đặt vấn đề.....	51
5.4.2 Giải pháp đề xuất	51

5.4.3 Kết quả đạt được.....	52
5.5 Giải pháp 4: Tích hợp trí tuệ nhân tạo với chi phí tối ưu.....	52
5.5.1 Đặt vấn đề.....	52
5.5.2 Giải pháp đề xuất.....	52
5.5.3 Kết quả đạt được.....	53
5.6 Giải pháp 5: Giao diện người dùng thích ứng với camera và điều kiện thực tế.....	53
5.6.1 Đặt vấn đề.....	53
5.6.2 Giải pháp đề xuất.....	53
5.6.3 Kết quả đạt được.....	54
5.7 Tổng kết chương	54
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	55
6.1 Kết luận.....	55
6.2 So sánh với các giải pháp hiện có.....	55
6.3 Hạn chế	56
6.4 Hướng phát triển	57
6.5 Tổng kết	57
6.6 Hướng dẫn viết đồ án.....	59
6.7 Hướng dẫn cài đặt.....	59
6.8 Tài liệu tham khảo	60
TÀI LIỆU THAM KHẢO	63
PHỤ LỤC	65
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP.....	65
A.1 Hướng dẫn cài đặt môi trường phát triển	65
A.1.1 Cài đặt công cụ phát triển backend.....	65
A.1.2 Cài đặt môi trường phát triển ứng dụng mobile	65

A.1.3 Cài đặt môi trường phát triển ứng dụng web	66
A.2 Cấu trúc thư mục project	66
A.3 Danh sách các API endpoints	66
A.4 Kịch bản kiểm thử chi tiết.....	67
B. ĐẶC TẢ USE CASE	68
B.1 Đặc tả chi tiết template JSON cho OMR	68
B.2 Thuật toán nhận diện bubble chi tiết	69
B.3 Cấu hình AMC cho đề thi trắc nghiệm	69
B.4 Danh sách các thành viên trong nhóm	70
B.5 Bảng phân công công việc	70

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống SMART GRADING	9
Hình 3.1	Lưu đồ quy trình xử lý OMR trong hệ thống SMART GRADING	25
Hình 3.2	Kiến trúc tổng quan của ứng dụng web SMART GRADING	27
Hình 3.3	Kiến trúc backend của hệ thống SMART GRADING	29
Hình 3.4	Lưu đồ tích hợp AMC trong hệ thống SMART GRADING .	30
Hình 4.1	Kiến trúc tổng quan hệ thống SMART GRADING với ba thành phần chính	32
Hình 4.2	Sơ đồ ERD của hệ thống SMART GRADING với các collections chính	38
Hình 4.3	Mockup một số màn hình chính của ứng dụng di động SMART GRADING	39
Hình 4.4	Lưu đồ chi tiết luồng chấm điểm trên thiết bị di động	42
Hình 4.5	Biểu đồ so sánh độ chính xác OMR theo nguồn ảnh và điều kiện chụp	45
Hình 4.6	Sơ đồ triển khai hệ thống SMART GRADING với các thành phần infrastructure	46
Hình 5.1	Lưu đồ pipeline tạo đề thi với AMC tích hợp sâu	49
Hình 5.2	Lưu đồ hệ thống AI proxy với caching và queueing	53

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case UC001 - Quét phiếu OMR	12
Bảng 2.2	Đặc tả use case UC002 - Tạo đề thi tự động	14
Bảng 2.3	Đặc tả use case UC003 - Xem báo cáo AI	15
Bảng 2.4	Đặc tả use case UC004 - Nộp đơn phúc tra	17
Bảng 2.5	Đặc tả use case UC005 - Quản lý kết quả thi	18
Bảng 2.6	Đặc tả use case UC006 - Đăng nhập hệ thống	20
Bảng 2.7	Đặc tả use case UC007 - Tạo nhóm gia đình	21
Bảng 2.8	Các yêu cầu phi chức năng của hệ thống	23
Bảng 4.1	Bảng tổng hợp công cụ phát triển	35
Bảng 4.2	Các trường được đánh index trong MongoDB	38
Bảng 4.3	Các màn hình chính của ứng dụng di động	40
Bảng 4.4	Các màn hình chính của ứng dụng web	40
Bảng 4.5	Các thông số kỹ thuật của hệ thống SMART GRADING . .	43
Bảng 4.6	Kết quả kiểm thử nhận diện OMR trên 1.000 phiếu thi . . .	44
Bảng 4.7	Một số test case mẫu cho các module quan trọng	45
Bảng 4.8	Tổng hợp kết quả load testing với các mức concurrent users khác nhau	47
Bảng 6.1	So sánh SMART GRADING với các giải pháp tương tự . .	56
Bảng 6.2	Các yêu cầu phần cứng và phần mềm	59

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AMC	Auto Multiple Choice - Công cụ tạo đề thi trắc nghiệm tự động bằng LaTeX
API	Application Programming Interface - Giao diện lập trình ứng dụng
BLoC	Business Logic Component - Thành phần xử lý logic nghiệp vụ
CNN	Convolutional Neural Network - Mạng nơ-ron tích chập
CNTT	Công nghệ thông tin
CRUD	Create, Read, Update, Delete - Tạo, đọc, cập nhật, xóa
CSS	Cascading Style Sheets - Bảng kiểu xếp chồng
Docker	Docker - Nền tảng đóng gói ứng dụng dạng container
DPI	Dots Per Inch - Số chấm trên inch
HTML	HyperText Markup Language - Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol - Giao thức truyền siêu văn bản
HTTPS	HyperText Transfer Protocol Secure - Giao thức truyền siêu văn bản bảo mật
IoT	Internet of Things - Internet vạn vật
JSON	JavaScript Object Notation - Định dạng dữ liệu theo cấu trúc đối tượng JavaScript
JWT	JSON Web Token - Mã thông báo web JSON
MVC	Model-View-Controller - Mô hình ba lớp Mô hình - Giao diện - Điều khiển
NoSQL	Not Only SQL - Không chỉ SQL
OCR	Optical Character Recognition - Nhận diện ký tự quang học

Thuật ngữ	Ý nghĩa
OMR	Optical Mark Recognition - Nhận diện vạch quang học
OTP	One-Time Password - Mật khẩu dùng một lần
PM2	Process Manager 2 - Trình quản lý tiến trình Node.js
RBAC	Role-Based Access Control - Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer - Kiến trúc chuyển trạng thái đại diện
SaaS	Software as a Service - Phần mềm như dịch vụ
SDK	Software Development Kit - Bộ công cụ phát triển phần mềm
SQL	Structured Query Language - Ngôn ngữ truy vấn có cấu trúc
SSH	Secure Shell - Giao thức truy cập từ xa bảo mật
SSL	Secure Sockets Layer - Tầng ổ bảo mật giao thức
UI	User Interface - Giao diện người dùng
URL	Uniform Resource Locator - Địa chỉ tài nguyên thống nhất
UUID	Universal Unique Identifier - Định danh duy nhất phổ quát
UX	User Experience - Trải nghiệm người dùng
ĐATN	Đồ án tốt nghiệp

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh giáo dục tại Việt Nam, hình thức thi trắc nghiệm đã và đang được áp dụng rộng rãi ở hầu hết các cấp học và bậc đào tạo. Từ các kỳ thi quan trọng như kỳ thi Trung học Phổ thông Quốc gia (THPTQG) với hàng triệu thí sinh tham dự, đến các bài kiểm tra định kỳ tại trường đại học với hàng trăm sinh viên, hình thức thi trắc nghiệm mang lại nhiều ưu điểm vượt trội so với thi tự luận [1]. Khả năng đánh giá khách quan, tốc độ xử lý nhanh chóng và tính nhất quán trong việc áp dụng tiêu chuẩn chấm điểm là những lợi thế nổi bật của phương thức thi này. Tuy nhiên, quy trình chấm thi trắc nghiệm truyền thống, đặc biệt tại các trường phổ thông và đại học với số lượng thí sinh lớn, vẫn đang đối mặt với những thách thức nghiêm trọng về mặt thời gian, chi phí và chất lượng.

Việc chấm thi trắc nghiệm bằng tay đòi hỏi nguồn nhân lực đáng kể. Theo ước tính thực tế, một giáo viên có thể chấm thủ công khoảng 80 đến 120 bài thi trắc nghiệm trong một giờ làm việc, và con số này giảm đáng kể khi khối lượng bài thi lên đến hàng nghìn hoặc hàng chục nghìn bài như trong các kỳ thi quy mô lớn. Khối lượng công việc khổng lồ này không chỉ gây ra tình trạng quá tải cho giáo viên mà còn làm tăng đáng kể nguy cơ sai sót do mệt mỏi và sự tập trung suy giảm theo thời gian. Các nghiên cứu trong lĩnh vực tâm lý học nhận thức đã chỉ ra rằng năng lực tập trung của con người suy giảm đáng kể sau 90 phút làm việc liên tục với các tác vụ lặp đi lặp lại, điều này trực tiếp ảnh hưởng đến chất lượng và tính chính xác của việc chấm thi thủ công.

Bên cạnh thách thức về thời gian và nhân lực, vấn đề chống gian lận trong thi trắc nghiệm cũng đặt ra những yêu cầu ngày càng cao về sự đa dạng của các mã đề. Khi thí sinh ngồi cạnh nhau trong phòng thi, khả năng các em nhìn bài của nhau hoặc trao đổi đáp án là rất cao nếu tất cả thí sinh cùng làm một đề duy nhất. Do đó, việc tạo ra nhiều mã đề khác nhau với thứ tự câu hỏi và đáp án được xáo trộn là biện pháp cần thiết, nhưng đồng thời lại làm tăng gấp bội khối lượng công việc cho khâu chấm thi. Mỗi mã đề lại cần một đáp án riêng và một người chấm riêng, tạo ra một vòng xoáy phức tạp về mặt tổ chức và điều phối.

Trên thị trường hiện nay, đã có một số giải pháp hỗ trợ chấm thi trắc nghiệm bằng công nghệ OMR (Optical Mark Recognition). Các giải pháp thương mại như Remark Office OMR hay GradeCam cung cấp đầy đủ tính năng nhưng đòi hỏi chi phí license cao và thường gắn liền với phần cứng scanner vật lý, khiến việc triển khai trở nên khó khăn đối với các trường học với ngân sách hạn chế. Đối với các

giải pháp phần mềm độc lập hoặc dịch vụ đám mây, mặc dù bỏ qua chi phí phần cứng, nhưng hầu hết đều yêu cầu kết nối internet liên tục để tải ảnh lên server xử lý, điều này hoàn toàn không khả thi trong các phòng thi thực tế tại nhiều trường học ở vùng nông thôn hoặc khu vực có hạ tầng mạng không ổn định. Các công cụ mã nguồn mở như OMRChecker tuy miễn phí nhưng giao diện dòng lệnh phức tạp, đòi hỏi kiến thức kỹ thuật chuyên sâu và hầu như không có giao diện đồ họa thân thiện, gây rào cản lớn cho giáo viên và nhân viên hành chính trong việc sử dụng.

Từ những phân tích trên, bài toán đặt ra là cần xây dựng một hệ thống chấm thi trắc nghiệm tự động đáp ứng đồng thời các yêu cầu về tính chính xác cao, chi phí triển khai thấp, khả năng hoạt động offline, hỗ trợ đa nền tảng (mobile và web), và đặc biệt là giao diện thân thiện để sử dụng cho mọi đối tượng người dùng từ giáo viên đến học sinh [2].

1.2 Mục tiêu và phạm vi đề tài

Để giải quyết các vấn đề đã nêu, nghiên cứu đã khảo sát và đánh giá tổng quan các giải pháp OMR hiện có trên thị trường. Các giải pháp được phân loại theo ba nhóm chính gồm phần mềm thương mại độc quyền, các công cụ mã nguồn mở, và các nền tảng SaaS dựa trên đám mây. Nhóm phần mềm thương mại như Remark Office OMR hay GradeCam cung cấp bộ tính năng đầy đủ bao gồm quản lý đề thi, nhận diện phiếu, và xuất báo cáo, tuy nhiên chi phí license hàng năm có thể lên đến hàng chục triệu đồng cho mỗi trường, chưa kể yêu cầu về thiết bị scanner chuyên dụng với giá thành từ vài triệu đến hàng chục triệu đồng. Nhóm công cụ mã nguồn mở như OMRChecker, Auto Multiple Choice (AMC) hay bộ công cụ dựa trên OpenCV cung cấp khả năng nhận diện OMR miễn phí nhưng đòi hỏi kiến thức kỹ thuật cao để vận hành, thiếu giao diện người dùng đồ họa, và phần lớn không hỗ trợ quy trình tạo đề thi đa mã đề một cách tự động và liền mạch. Nhóm giải pháp SaaS như ScanTa hay các nền tảng tương tự hoạt động trên nền tảng đám mây với giao diện web thân thiện, nhưng yêu cầu kết nối internet liên tục và thường áp dụng mô hình tính phí theo số lượt quét hoặc theo tháng, dẫn đến chi phí vận hành lâu dài cao.

Dựa trên những hạn chế của các giải pháp hiện có, đề tài đặt ra các mục tiêu cụ thể như sau. Mục tiêu thứ nhất là xây dựng một hệ thống chấm thi tự động sử dụng công nghệ OMR, cho phép nhận diện và chấm điểm phiếu trắc nghiệm một cách nhanh chóng và chính xác. Mục tiêu thứ hai là hệ thống phải hỗ trợ đa nền tảng gồm ứng dụng di động trên hệ điều hành iOS và Android cùng ứng dụng web trên trình duyệt, đảm bảo khả năng tiếp cận rộng rãi cho mọi đối tượng người dùng mà không phụ thuộc vào thiết bị cụ thể. Mục tiêu thứ ba là đạt độ chính xác nhận

diện tối thiểu 95% đối với các ảnh chụp phiếu trắc nghiệm từ camera smartphone trong điều kiện ánh sáng thông thường. Mục tiêu thứ tư là thời gian xử lý nhận diện và chấm điểm mỗi phiếu không quá ba giây trên thiết bị di động tầm trung, đảm bảo trải nghiệm người dùng mượt mà. Mục tiêu thứ năm là hệ thống hỗ trợ tạo và quản lý đề thi với nhiều mã đề khác nhau, trong đó mỗi mã đề có thứ tự câu hỏi và đáp án được xáo trộn tự động để phòng chống gian lận thi cử. Mục tiêu thứ sáu là tích hợp mô-đun trí tuệ nhân tạo để phân tích kết quả thi, xác định các điểm yếu của học sinh và đề xuất câu hỏi luyện tập phù hợp. Mục tiêu thứ bảy là thiết kế kiến trúc offline-first cho ứng dụng di động, cho phép hoạt động đầy đủ trong môi trường không có kết nối internet và tự động đồng bộ dữ liệu khi có mạng. Mục tiêu cuối cùng là triển khai hệ thống với chi phí thấp, ưu tiên sử dụng các công nghệ mã nguồn mở và miễn phí để hạ thấp rào cản tiếp cận cho các trường học.

Phạm vi nghiên cứu và phát triển của đề tài tập trung vào bài toán thi trắc nghiệm với hình thức lựa chọn một đáp án đúng trong bốn phương án A, B, C, D, phù hợp với hình thức thi phổ biến tại các trường trung học phổ thông và đại học ở Việt Nam. Mỗi phiếu thi có thể chứa từ 10 đến 50 câu hỏi, với các thông tin bổ sung về mã thí sinh và mã đề được đánh dấu trên cùng phiếu. Hệ thống được thiết kế cho quy mô sử dụng từ một lớp học nhỏ đến toàn bộ trường học, với khả năng xử lý đồng thời nhiều bài thi từ các kỳ thi khác nhau. Về đối tượng người dùng, hệ thống phục vụ ba nhóm chính gồm giáo viên sử dụng web để tạo đề và quản lý kết quả, học sinh sử dụng ứng dụng di động để quét phiếu và xem điểm, và quản trị viên quản lý người dùng và cấu hình hệ thống.

1.3 Định hướng giải pháp

Từ các mục tiêu đã xác định, đề tài lựa chọn hướng tiếp cận xây dựng hệ thống phần mềm phân tán đa nền tảng, kết hợp công nghệ nhận diện ảnh OMR và trí tuệ nhân tạo, với ba thành phần chính tương ứng với ba tầng của kiến trúc ứng dụng hiện đại.

Về nền tảng phát triển ứng dụng di động, đề tài sử dụng Flutter, một framework cross-platform được phát triển bởi Google, cho phép xây dựng ứng dụng native cho cả iOS và Android từ một codebase duy nhất bằng ngôn ngữ lập trình Dart. Lý do lựa chọn Flutter bao gồm ba yếu tố chính. Thứ nhất, khả năng cross-platform giúp giảm đáng kể chi phí phát triển và bảo trì so với việc xây dựng hai ứng dụng riêng biệt cho iOS và Android. Thứ hai, Flutter sử dụng engine Skia để vẽ giao diện trực tiếp trên thiết bị, cho hiệu suất cao hơn so với các framework cross-platform khác sử dụng WebView hoặc bridge trung gian. Thứ ba, hệ sinh thái phong phú của Flutter với các gói thư viện hỗ trợ xử lý ảnh và camera giúp đẩy nhanh quá trình

phát triển. Để quản lý trạng thái ứng dụng, hệ thống áp dụng mẫu thiết kế BLoC (Business Logic Component), cho phép tách biệt hoàn toàn logic nghiệp vụ khỏi giao diện người dùng, tăng khả năng kiểm thử và tái sử dụng mã nguồn.

Về nền tảng phát triển ứng dụng web, đề tài sử dụng React phiên bản 19 kết hợp với ngôn ngữ TypeScript và công cụ build Vite. React là thư viện JavaScript được phát triển bởi Meta, với ưu điểm nổi bật là mô hình component-based cho phép tái sử dụng giao diện cao và cộng đồng phát triển đông đảo với hàng triệu thư viện mã nguồn mở. TypeScript bổ sung hệ thống kiểu tĩnh giúp phát hiện lỗi sớm trong quá trình phát triển và cải thiện chất lượng mã nguồn. Vite được chọn làm công cụ build thay cho webpack truyền thống nhờ tốc độ khởi động và hot module replacement nhanh hơn đáng kể, đặc biệt quan trọng trong môi trường phát triển với dự án có quy mô lớn. State management cho ứng dụng web sử dụng Zustand, một thư viện nhẹ với API đơn giản nhưng đủ mạnh mẽ để quản lý trạng thái phức tạp của ứng dụng.

Về kiến trúc phía backend, đề tài lựa chọn Node.js với framework Express và cơ sở dữ liệu MongoDB. Node.js là runtime JavaScript phía server cho phép xây dựng ứng dụng mạng với hiệu suất cao nhờ mô hình non-blocking I/O, đặc biệt phù hợp với các ứng dụng cần xử lý nhiều kết nối đồng thời như hệ thống API. MongoDB là cơ sở dữ liệu NoSQL document-oriented, cho phép lưu trữ dữ liệu phức tạp như cấu trúc câu hỏi, đáp án và kết quả thi dưới dạng document JSON một cách tự nhiên và linh hoạt. Mongoose được sử dụng như ODM (Object Data Modeling) để định nghĩa schema, kiểm tra dữ liệu và tạo các middleware cho các thao tác trên database. Để tạo đề thi đa mã đề, hệ thống tích hợp công cụ AMC (Auto Multiple Choice), một phần mềm mã nguồn mở chạy trên LaTeX, cho phép sinh tự động nhiều phiên bản đề thi với câu hỏi và đáp án được xáo trộn, đồng thời xuất ra tọa độ OMR chính xác phục vụ cho quá trình quét và nhận diện.

Về mô-đun nhận diện OMR, đề tài triển khai engine nhận diện trực tiếp trên thiết bị di động (client-side processing) thay vì gửi ảnh lên server. Lợi ích của phương pháp này là thời gian phản hồi tức thì, không phụ thuộc vào tốc độ mạng, và hoạt động được ngay cả khi không có kết nối internet. Engine nhận diện sử dụng các kỹ thuật xử lý ảnh bao gồm tiền xử lý ảnh (chuyển đổi grayscale, cân bằng histogram, khử nhiễu Gaussian), phát hiện góc phiếu bằng thuật toán Canny edge detection và contour analysis, phép biến đổi phối cảnh (perspective transform) để chuẩn hóa phiếu, và phân tích mật độ pixel trong từng vùng bubble để xác định đáp án được tô. Điểm mấu chốt của giải pháp là sử dụng template JSON chứa tọa độ chính xác của từng bubble, được sinh tự động bởi AMC CLI khi compile đề thi, đảm bảo độ

chính xác tuyệt đối không phụ thuộc vào phép tính toán thủ công.

Về mô-đun trí tuệ nhân tạo, đề tài tích hợp Google Gemini API làm nền tảng chính cho các chức năng sinh câu hỏi tự động, phân tích lỗi sai của học sinh và chatbot hỗ trợ ôn tập. Gemini được lựa chọn nhờ khả năng xử lý ngữ cảnh dài, hiệu suất vượt trội trên các benchmark trong lĩnh vực suy luận và tạo sinh văn bản, cùng mô hình pricing hợp lý cho các ứng dụng có lượng sử dụng vừa phải.

Hệ thống được đặt tên là SMART GRADING, viết tắt của System for Multi-Platform Automated Recognition and Testing with Grading, Intelligent Notifications, and Detailed Analytics.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau. Chương 2 trình bày kết quả khảo sát hiện trạng quy trình chấm thi trắc nghiệm tại các trường phổ thông và đại học, phân tích chi tiết các giải pháp OMR hiện có trên thị trường về ưu điểm và hạn chế của từng giải pháp, đồng thời đặc tả đầy đủ các yêu cầu chức năng và phi chức năng của hệ thống thông qua biểu đồ use case, quy trình nghiệp vụ và các bảng đặc tả chi tiết cho các use case quan trọng nhất.

Trong Chương 3, em trình bày các nền tảng lý thuyết và công nghệ được sử dụng trong đề tài, bao gồm công nghệ xử lý ảnh OMR, framework phát triển ứng dụng di động Flutter, thư viện giao diện web React, kiến trúc backend Node.js, cơ sở dữ liệu MongoDB, và các công cụ hỗ trợ khác. Mỗi công nghệ được phân tích theo ba khía cạnh gồm giới thiệu tổng quan, so sánh với các lựa chọn thay thế, và lý do lựa chọn cụ thể cho đề tài này.

Chương 4 trình bày chi tiết quá trình phân tích và thiết kế hệ thống, bao gồm kiến trúc tổng quan theo mô hình ba lớp, biểu đồ các gói và biểu đồ lớp UML, thiết kế cơ sở dữ liệu MongoDB với sơ đồ ERD, thiết kế giao diện người dùng cho cả ứng dụng mobile và web, và kết quả triển khai thực tế bao gồm các số liệu thống kê về mã nguồn, kiểm thử và triển khai hệ thống.

Chương 5 mô tả các giải pháp kỹ thuật nổi bật và đóng góp chính của đề tài, tập trung vào năm giải pháp cốt lõi gồm pipeline tạo đề thi tự động với AMC, engine nhận diện OMR chạy trên thiết bị di động, kiến trúc offline-first cho ứng dụng di động, tích hợp trí tuệ nhân tạo để phân tích kết quả học tập, và giao diện camera thích ứng với điều kiện thực tế. Đây là chương quan trọng nhất để đánh giá năng lực giải quyết vấn đề và khả năng áp dụng kiến thức vào thực tiễn của sinh viên.

Chương 6 tổng kết các kết quả đạt được của đề tài, so sánh với các mục tiêu đã đề ra ban đầu và với các giải pháp tương tự trên thị trường, đồng thời đề xuất các

hướng phát triển tiếp theo để hoàn thiện và mở rộng hệ thống trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Khảo sát hiện trạng

Để hiểu rõ bối cảnh và nhu cầu thực tế của việc chấm thi trắc nghiệm tại các cơ sở giáo dục Việt Nam, nhóm nghiên cứu đã tiến hành khảo sát thực tế tại ba trường đại học và hai trường trung học phổ thông trên địa bàn thành phố Hà Nội trong giai đoạn từ tháng 9 đến tháng 12 năm 2025. Quy trình khảo sát và phân tích yêu cầu được thực hiện theo phương pháp luận kỹ nghệ phần mềm [2], [3]. Mục tiêu của cuộc khảo sát là thu thập thông tin về quy trình chấm thi hiện tại, các khó khăn và thách thức mà giáo viên và nhân viên đang gặp phải, cũng như đánh giá nhu cầu và mong muốn đối với một giải pháp hỗ trợ chấm thi tự động. Phương pháp khảo sát kết hợp giữa phỏng vấn sâu với các giáo viên có kinh nghiệm chấm thi và bảng câu hỏi khảo sát với quy mô mẫu lớn hơn để có cái nhìn đa chiều về thực trạng, theo cách tiếp cận đánh giá lớp học đã được Black và Wiliam đề xuất [1].

Kết quả khảo sát cho thấy 100% các trường được khảo sát đều sử dụng hình thức thi trắc nghiệm cho ít nhất một số môn học, trong đó các môn toán, vật lý, hóa học, sinh học và ngoại ngữ là những môn có tỷ lệ thi trắc nghiệm cao nhất. Đối với các kỳ thi quy mô lớn như thi giữa kỳ và cuối kỳ, trung bình mỗi trường đại học tổ chức từ 15 đến 30 môn thi trắc nghiệm mỗi học kỳ, với tổng số bài thi dao động từ 5.000 đến 15.000 bài thi mỗi đợt thi. Tại các trường trung học phổ thông, con số này thấp hơn với khoảng 3 đến 8 môn thi trắc nghiệm mỗi học kỳ và tổng số bài thi từ 2.000 đến 8.000 bài. Thời gian chấm thi trung bình được ghi nhận là từ 2 đến 5 ngày làm việc cho mỗi đợt thi, trong đó khối lượng lớn nhất công việc tập trung vào khâu kiểm tra điểm thi trùng lặp, nhập liệu kết quả và xử lý các trường hợp đặc biệt như bài thi bị tô nhầm hoặc thiếu thông tin.

Về phương thức chấm thi hiện tại, kết quả khảo sát cho thấy 60% các trường vẫn sử dụng phương pháp chấm tay hoàn toàn, trong đó giáo viên đọc đáp án và đánh dấu từng câu trên phiếu trả lời bằng bút chì hoặc bút đánh dấu. Phương pháp này tuy đơn giản nhưng tốc độ chậm và tỷ lệ sai sót cao, đặc biệt khi giáo viên phải chấm liên tục trong nhiều giờ. 25% các trường sử dụng các công cụ hỗ trợ đơn giản như bảng đục lỗ hoặc com-pa chấm điểm, cho phép đánh dấu nhanh hơn nhưng vẫn đòi hỏi thao tác thủ công và không thể tránh khỏi sai sót do mệt mỏi. Chỉ 15% các trường được khảo sát đã triển khai hệ thống chấm thi tự động bằng máy quét OMR chuyên dụng, tuy nhiên các hệ thống này đều gặp vấn đề về chi phí bảo trì cao, cần nhân viên kỹ thuật vận hành, và hạn chế trong việc xử lý các tình huống đặc biệt như phiếu bị nhàu hoặc tô không đúng vị trí.

Một phát hiện quan trọng từ cuộc khảo sát là 85% giáo viên được hỏi bày tỏ mong muốn được sử dụng một công cụ chấm thi tự động trên smartphone, vì điện thoại thông minh là thiết bị mà hầu hết giáo viên đã sẵn có và sử dụng thành thạo. Yêu cầu quan trọng nhất được đặt ra là tính dễ sử dụng, với 92% người được hỏi nhấn mạnh rằng giao diện phải đơn giản và trực quan, không đòi hỏi đào tạo hoặc hướng dẫn nhiều. 78% giáo viên đặc biệt quan tâm đến khả năng hoạt động offline của ứng dụng, vì nhiều trường học có kết nối internet không ổn định hoặc phòng thi nằm ở các khu vực xa với hạ tầng mạng kém. Ngoài ra, 70% giáo viên cho biết họ thường xuyên cần tạo nhiều mã đề khác nhau để phòng chống gian lận, và 65% mong muốn hệ thống có khả năng xuất kết quả ra file Excel hoặc PDF để phục vụ công tác báo cáo.

Kết quả khảo sát tại hai trường trung học phổ thông cho thấy nhu cầu đặc thù về phía học sinh và phụ huynh. Cụ thể, 80% phụ huynh được khảo sát cho biết họ muốn nhận kết quả thi của con em mình qua điện thoại hoặc email ngay sau khi kỳ thi kết thúc, thay vì phải chờ đợi từ 3 đến 7 ngày như hiện tại. 60% học sinh được hỏi bày tỏ mong muốn có thể xem lại bài thi của mình sau khi có kết quả, đặc biệt là những câu mà mình bị sai, để tự ôn tập và cải thiện [4]. Đây là những nhu cầu hợp lý và hệ thống cần được thiết kế để đáp ứng đầy đủ.

Trong quá trình nghiên cứu, nhóm đã khảo sát và phân tích chi tiết các giải pháp hỗ trợ chấm thi trắc nghiệm bằng công nghệ OMR hiện có trên thị trường, bao gồm cả các sản phẩm thương mại và mã nguồn mở. Phương pháp nhận dạng vạch đánh dấu (Optical Mark Recognition) đã được nghiên cứu từ lâu và ứng dụng rộng rãi trong nhiều hệ thống nhận dạng tài liệu khác nhau [5]. Kết quả phân tích cho thấy Remark Office OMR là giải pháp thương mại hàng đầu với độ chính xác rất cao và bộ tính năng phong phú, tuy nhiên chi phí license hàng năm có thể lên đến hàng chục triệu đồng và yêu cầu thiết bị scanner chuyên dụng. GradeCam cung cấp giải pháp quét bằng camera smartphone thông qua ứng dụng web, cho phép sử dụng ngay mà không cần mua scanner, nhưng yêu cầu đăng ký tài khoản trả phí và dữ liệu được xử lý trên đám mây. OMRChecker là công cụ mã nguồn mở dựa trên Python và OpenCV [6], cho phép nhận diện OMR miễn phí nhưng giao diện dòng lệnh phức tạp, không có giao diện đồ họa. AMC là công cụ mã nguồn mở tốt nhất cho việc tạo đề thi trắc nghiệm với khả năng sinh nhiều mã đề và xuất tọa độ OMR chính xác, tuy nhiên đây chỉ là công cụ dòng lệnh chạy trên LaTeX, không có thành phần ứng dụng web hoặc di động đi kèm.

Từ cuộc khảo sát và phân tích, nhóm rút ra ba bài học chính. Thứ nhất, giải pháp lý tưởng cần kết hợp khả năng tạo đề thi đa mã đề của AMC với ứng dụng quét trên smartphone và hệ thống quản lý kết quả trên web. Thứ hai, việc xử lý

OMR trên thiết bị di động (edge computing) là xu hướng tất yếu trong bối cảnh smartphone ngày càng mạnh và nhu cầu về tính di động ngày càng cao. Thứ ba, kiến trúc offline-first là yêu cầu bắt buộc đối với các trường học tại Việt Nam, nơi hạ tầng mạng chưa đồng đều giữa các vùng miền.

2.2 Tổng quan chức năng

Phần này trình bày tổng quan các chức năng chính của hệ thống SMART GRADING, được mô hình hóa thông qua biểu đồ use case tổng quát. Hệ thống bao gồm năm actor chính là học sinh, giáo viên, quản trị viên trường, quản trị viên hệ thống và phụ huynh, cùng với các use case được nhóm theo các gói chức năng tương ứng. Phương pháp mô hình hóa use case được áp dụng tuân theo nguyên lý phân tích yêu cầu hướng đối tượng đã được Sommerville trình bày [2], đồng thời vận dụng các mẫu thiết kế phần mềm phổ biến của Gamma và cộng sự [7] để tổ chức cấu trúc hệ thống.

[Hình minh họa: Biểu đồ use case tổng quát]

Figure 2.1: Biểu đồ use case tổng quát của hệ thống SMART GRADING

Hình 2.1 thể hiện biểu đồ use case tổng quát với các actor và các gói chức năng chính. Nhóm chức năng quản lý người dùng bao gồm các use case cho phép đăng ký tài khoản mới với xác thực email, đăng nhập bằng email và mật khẩu hoặc qua OTP gửi đến số điện thoại, quản lý thông tin cá nhân và đổi mật khẩu, phân quyền người dùng theo vai trò với bốn cấp độ gồm quản trị hệ thống, quản trị trường, giáo viên và học sinh, cùng với chức năng liên kết tài khoản phụ huynh với tài khoản học sinh để nhận thông báo kết quả thi.

Nhóm chức năng quản lý đề thi cho phép giáo viên tạo đề thi mới với các thông tin bao gồm tiêu đề, môn học, thời gian làm bài, số câu hỏi và thời gian bắt đầu và kết thúc. Giáo viên có thể nhập câu hỏi thủ công từng câu một hoặc sử dụng chức năng sinh câu hỏi tự động từ AI. Hệ thống hỗ trợ hai phương thức tạo mã đề gồm tạo thủ công với giáo viên tự thiết lập số lượng và thứ tự đáp án cho từng mã đề, và tạo tự động bằng AMC CLI trong đó hệ thống tự sinh N mã đề với câu hỏi và đáp án được xáo trộn theo thuật toán ngẫu nhiên có kiểm soát, đảm bảo mỗi mã đề có độ khó tương đương nhau. Sau khi tạo đề, hệ thống cho phép xem trước, tải về file PDF để in ấn, và gửi thông báo đến học sinh về kỳ thi sắp tới.

Nhóm chức năng quét và nhận diện OMR cho phép giáo viên sử dụng camera trên smartphone để chụp ảnh phiếu trả lời trắc nghiệm. Ứng dụng thực hiện các bước xử lý bao gồm tiền xử lý ảnh với các kỹ thuật cân bằng ánh sáng, khử méo hình và chuẩn hóa kích thước, phát hiện vùng phiếu trả lời bằng thuật toán phát hiện cạnh Canny và tìm contour, áp dụng phép biến đổi phối cảnh để chỉnh ảnh nghiêng thành ảnh nhìn thẳng, và phân tích mật độ pixel trong từng vùng bubble để xác định đáp án được tô. Kết quả bao gồm điểm số, số câu đúng sai, và hình ảnh phiếu đã được đánh dấu vùng bubble. Giáo viên có thể xem lại kết quả, chỉnh sửa nếu phát hiện sai sót, và xác nhận để lưu vào hệ thống.

Nhóm chức năng quản lý kết quả thi cho phép giáo viên xem danh sách kết quả theo từng kỳ thi với các thông tin về điểm số, thống kê phân bố điểm, và danh sách học sinh có kết quả bất thường. Giáo viên có thể lọc và tìm kiếm kết quả theo nhiều tiêu chí như tên học sinh, mã học sinh, mã đề, và khoảng điểm. Hệ thống hỗ trợ xuất kết quả ra file Excel với đầy đủ thông tin và định dạng phù hợp cho việc báo cáo. Giáo viên có thể tạo báo cáo tổng hợp theo mẫu có sẵn hoặc tự thiết kế với các trường dữ liệu và biểu đồ tùy chọn.

Nhóm chức năng phúc tra và khiếu nại cho phép học sinh xem lại bài thi của mình sau khi có kết quả, trong đó hiển thị rõ từng câu hỏi cùng đáp án đúng và đáp án mà học sinh đã chọn. Học sinh có thể nộp đơn phúc tra nếu cho rằng đáp án bị nhận diện sai, kèm theo lý do và minh chứng. Giáo viên tiếp nhận và xem xét đơn phúc tra, có thể chấm lại phiếu hoặc điều chỉnh kết quả nếu xác nhận có sai sót.

Nhóm chức năng trí tuệ nhân tạo tích hợp các tính năng gồm sinh câu hỏi tự động từ nội dung tài liệu hoặc chủ đề do giáo viên cung cấp, phân tích kết quả thi để xác định các chủ đề mà học sinh còn yếu dựa trên tần suất trả lời sai theo từng chủ đề, đề xuất câu hỏi luyện tập phù hợp với mức độ khó của học sinh, và chatbot trả lời câu hỏi liên quan đến bài giảng và nội dung môn học.

Nhóm chức năng thông báo và theo dõi cho phép gửi thông báo đến học sinh và phụ huynh qua notification trong ứng dụng và qua SMS hoặc email khi có kết quả thi mới, khi có thông báo từ giáo viên, hoặc khi có cập nhật về đơn phúc tra. Phụ huynh có thể theo dõi lịch thi và kết quả học tập của con em thông qua ứng dụng dành cho phụ huynh.

Quy trình nghiệp vụ chính của hệ thống SMART GRADING bao gồm năm quy trình quan trọng. Quy trình thứ nhất là quy trình tạo đề thi bắt đầu khi giáo viên đăng nhập vào hệ thống web, nhập thông tin kỳ thi, nhập hoặc sinh câu hỏi, thiết lập mã đề, compile bằng AMC, và tải về PDF để in ấn. Quy trình thứ hai là quy trình tổ chức thi trong đó giáo viên phát phiếu cho học sinh, học sinh làm bài, và

giáo viên thu phiếu. Quy trình thứ ba là quy trình chấm thi trong đó giáo viên mở ứng dụng mobile, chọn kỳ thi, chụp ảnh từng phiếu, xem kết quả và xác nhận, sau đó dữ liệu được đồng bộ lên server. Quy trình thứ tư là quy trình phúc tra trong đó học sinh xem kết quả, nộp đơn phúc tra nếu có sai sót, giáo viên xem xét và quyết định, và kết quả được cập nhật nếu có thay đổi. Quy trình thứ năm là quy trình báo cáo AI trong đó giáo viên chọn kỳ thi, hệ thống gọi Gemini API để phân tích, và báo cáo được hiển thị và có thể tải về.

2.2.1 Đặc tả use case Quét phiếu OMR

Mục	Nội dung		
Mã UC (UC #)	UC001		
Tên usecase	Quét phiếu OMR		
Tác nhân	Giáo viên (Teacher)		
Điều kiện trước	Giáo viên đã đăng nhập vào hệ thống và có quyền quét bài thi. Đã có ít nhất một kỳ thi đang mở trong hệ thống. Thiết bị di động đã được cấp quyền truy cập camera.		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Chọn chức năng “Quét phiếu OMR”.
	2	System	Hiển thị danh sách các kỳ thi đang mở để giáo viên chọn.
	3	User	Chọn kỳ thi cần chấm và nhấn “Bắt đầu quét”.
	4	System	Mở camera và hiển thị khung hình hướng dẫn căn phiếu.
	5	User	Cầm camera hướng vào phiếu trả lời và chờ hệ thống tự động phát hiện phiếu.
	6	System	Khi phát hiện phiếu ổn định trong khung hình, tự động chụp ảnh và tiến hành xử lý.
	7	System	Thực hiện tiền xử lý ảnh (cân bằng ánh sáng, khử méo, chuẩn hóa kích thước).
	8	System	Phát hiện vùng phiếu và áp dụng phép biến đổi phối cảnh để căn chỉnh.
	9	System	Nhận diện mã thí sinh, mã đề và đáp án của từng câu hỏi.

Mục	Nội dung		
	10	System	Tính điểm và hiển thị kết quả bao gồm điểm số, số câu đúng/sai, hình ảnh phiếu đã đánh dấu.
	11	User	Xem xét kết quả và nhấn “Xác nhận” để lưu hoặc “Chụp lại” để quét phiếu khác.
	12	System	Lưu kết quả vào cơ sở dữ liệu và đồng bộ lên server khi có kết nối mạng.
Luồng thực thi mở rộng	No.	Thực hiện	Hành động
	4a	System	Thông báo lỗi: Vui lòng cấp quyền truy cập camera cho ứng dụng.
	6a	System	Thông báo cảnh báo: Phiếu bị mờ hoặc thiếu sáng, vui lòng chụp lại.
	6b	System	Thông báo cảnh báo: Phát hiện tô đúp (double fill) ở câu X, yêu cầu giáo viên xử lý thủ công.
	9a	System	Thông báo lỗi: Không nhận diện được mã đề, vui lòng chụp lại toàn bộ phiếu.
	10a	System	Thông báo lỗi: Mã đề không khớp với danh sách mã đề của kỳ thi.
	11a	System	Cho phép giáo viên chỉnh sửa thủ công đáp án trước khi xác nhận.
	12a	System	Lưu kết quả vào bộ nhớ cục bộ và đồng bộ lên server khi có kết nối mạng.
Điều kiện sau	Kết quả chấm phiếu OMR được lưu thành công vào hệ thống. Học sinh và phụ huynh (nếu liên kết) nhận được thông báo kết quả thi. Thống kê kết quả của kỳ thi được cập nhật tự động.		

Table 2.1: Đặc tả use case UC001 - Quét phiếu OMR

2.2.2 Đặc tả use case Tạo đề thi tự động

Mục	Nội dung
Mã UC (UC #)	UC002
Tên usecase	Tạo đề thi tự động với AMC

Mục	Nội dung		
Tác nhân	Giáo viên (Teacher)		
Điều kiện trước	Giáo viên đã đăng nhập vào hệ thống web và có quyền tạo đề thi. Ngân hàng câu hỏi của giáo viên đã có sẵn danh sách câu hỏi hoặc giáo viên đã chuẩn bị file LaTeX.		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Chọn chức năng “Tạo đề thi tự động”.
	2	System	Hiển thị form nhập thông tin kỳ thi gồm tiêu đề, môn học, lớp, thời gian, số câu hỏi, số mã đề.
	3	User	Nhập thông tin kỳ thi và chọn phương thức tạo câu hỏi (nhập tay, sinh từ AI, hoặc tải file LaTeX).
	4	User	Tải lên file LaTeX chứa đề thi hoặc nhập danh sách câu hỏi từ ngân hàng câu hỏi.
	5	System	Validate định dạng file LaTeX và nội dung câu hỏi.
	6	User	Chọn số lượng mã đề cần sinh và nhấn “Tạo đề”.
	7	System	Gọi AMC CLI để compile đề LaTeX thành N file PDF với thứ tự câu hỏi và đáp án được xáo trộn.
	8	System	Trích xuất tọa độ OMR từ AMC output và lưu vào cơ sở dữ liệu.
	9	System	Hiển thị danh sách các mã đề đã tạo cùng preview từng mã đề.
	10	User	Xem xét và nhấn “Tải về tất cả” để tải file PDF hoặc chọn mã đề cụ thể để tải về.
	11	System	Đóng gói các file PDF thành file nén và cung cấp link tải về.
	12	System	Thông báo “Tạo đề thi thành công” và chuyển đến trang quản lý kỳ thi.
	No.	Thực hiện	Hành động

Mục	Nội dung		
	5a	System	Thông báo lỗi: File LaTeX không đúng định dạng, vui lòng kiểm tra lại.
	5b	System	Thông báo cảnh báo: Một số câu hỏi thiếu đáp án đúng, vui lòng bổ sung.
	7a	System	Thông báo lỗi: Quá trình compile AMC thất bại, vui lòng kiểm tra log để biết chi tiết.
	8a	System	Thông báo lỗi: Không trích xuất được tọa độ OMR, vui lòng liên hệ quản trị viên.
	11a	System	Thông báo lỗi: Không thể đóng gói file, vui lòng thử lại sau.
Điều kiện sau	Đề thi với N mã đề được tạo thành công, tọa độ OMR được lưu trong cơ sở dữ liệu. Giáo viên có thể tải về file PDF để in ấn và tổ chức kỳ thi.		

Table 2.2: Đặc tả use case UC002 - Tạo đề thi tự động

2.2.3 Đặc tả use case Xem báo cáo AI

Mục	Nội dung		
Mã UC (UC #)	UC003		
Tên usecase	Xem báo cáo AI phân tích kết quả thi		
Tác nhân	Giáo viên (Teacher), Học sinh (Student)		
Điều kiện trước	Actor đã đăng nhập vào hệ thống. Đã có ít nhất một kỳ thi đã được chấm xong với kết quả trong cơ sở dữ liệu. API Gemini đang hoạt động bình thường.		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Chọn chức năng “Báo cáo AI” từ menu.
	2	System	Hiển thị danh sách các kỳ thi đã hoàn thành để actor chọn.
	3	User	Chọn kỳ thi cần phân tích và nhấn “Tạo báo cáo”.
	4	System	Truy vấn dữ liệu kết quả thi và câu hỏi từ cơ sở dữ liệu.

Mục	Nội dung		
	5	System	Gọi Gemini API để phân tích dữ liệu và sinh báo cáo.
	6	System	Hiển thị báo cáo bao gồm biểu đồ phân bố điểm, bảng thống kê điểm yếu theo chủ đề, danh sách học sinh cần hỗ trợ, đề xuất câu hỏi luyện tập.
	7	User	Xem chi tiết từng phần của báo cáo và các biểu đồ thống kê.
	8	User	Chọn xem chi tiết của một học sinh cụ thể trong báo cáo.
	9	System	Hiển thị chi tiết điểm mạnh/yếu và lịch sử làm bài của học sinh đó.
	10	User	Nhấn “Tải báo cáo PDF” để tải xuống hoặc “Chia sẻ qua link” để gửi cho đồng nghiệp.
	11	System	Xuất báo cáo dạng PDF hoặc tạo link chia sẻ có thời hạn.
Luồng thực thi mở rộng	No.	Thực hiện	Hành động
	4a	System	Thông báo lỗi: Không tìm thấy dữ liệu kết quả của kỳ thi này.
	5a	System	Thông báo lỗi: Không thể kết nối tới Gemini API, vui lòng thử lại sau.
	5b	System	Thông báo cảnh báo: Quá trình phân tích mất nhiều thời gian, vui lòng chờ.
	10a	System	Thông báo lỗi: Không thể tạo file PDF, vui lòng thử lại.
	11a	System	Thông báo lỗi: Không thể tạo link chia sẻ, vui lòng thử lại sau.
Điều kiện sau	Báo cáo AI được hiển thị đầy đủ cho actor. Giáo viên có thể sử dụng thông tin phân tích để điều chỉnh phương pháp giảng dạy. Học sinh nhận được đề xuất luyện tập phù hợp với điểm yếu của mình.		

Table 2.3: Đặc tả use case UC003 - Xem báo cáo AI

2.2.4 Đặc tả use case Nộp đơn phúc tra

Mục	Nội dung		
Mã UC (UC #)	UC004		
Tên usecase	Nộp đơn phúc tra bài thi		
Tác nhân	Học sinh (Student), Giáo viên (Teacher) - phía xử lý		
Điều kiện trước	Học sinh đã đăng nhập vào hệ thống. Đã có kết quả thi của học sinh và đang trong thời hạn phúc tra (48 giờ sau khi có kết quả).		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Mở mục “Kết quả thi của tôi” từ menu chính.
	2	System	Hiển thị danh sách các bài thi đã có kết quả.
	3	User	Chọn bài thi muốn phúc tra và nhấn “Yêu cầu phúc tra”.
	4	System	Hiển thị chi tiết bài thi với từng câu hỏi, đáp án đúng và đáp án đã chọn.
	5	User	Chọn câu muốn phúc tra, nhập lý do phúc tra và có thể đính kèm minh chứng.
	6	User	Nhấn “Gửi đơn phúc tra”.
	7	System	Validate thông tin đơn phúc tra và lưu vào cơ sở dữ liệu.
	8	System	Gửi thông báo đến giáo viên phụ trách kỳ thi.
	9	Teacher	Xem xét đơn phúc tra, xem lại ảnh phiếu gốc và đưa ra quyết định.
	10	System	Cập nhật kết quả bài thi nếu có điều chỉnh và gửi thông báo kết quả phúc tra cho học sinh.
Luồng thực thi mở rộng	No.	Thực hiện	Hành động
	3a	System	Thông báo: Đã quá thời hạn phúc tra (sau 48 giờ), không thể gửi đơn.

Mục	Nội dung		
	5a	System	Thông báo lỗi: Vui lòng chọn ít nhất một câu hỏi để phúc tra.
	5b	System	Thông báo lỗi: Vui lòng nhập lý do phúc tra chi tiết.
	7a	System	Thông báo lỗi: Không thể gửi đơn phúc tra. Vui lòng thử lại sau.
	10a	System	Giáo viên có thể thêm nhận xét khi quyết định điều chỉnh hoặc giữ nguyên kết quả.
Điều kiện sau	Đơn phúc tra được xử lý và kết quả bài thi được cập nhật (nếu có thay đổi). Học sinh nhận được thông báo kết quả phúc tra. Lịch sử phúc tra được lưu lại để tra cứu.		

Table 2.4: Đặc tả use case UC004 - Nộp đơn phúc tra

2.2.5 Đặc tả use case Quản lý kết quả thi

Mục	Nội dung		
Mã UC (UC #)	UC005		
Tên usecase	Quản lý kết quả thi		
Tác nhân	Giáo viên (Teacher), Quản trị trường (School Admin)		
Điều kiện trước	Actor đã đăng nhập vào hệ thống web. Đã có ít nhất một kỳ thi với kết quả trong hệ thống. Actor có quyền xem kết quả trong phạm vi được phân công.		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Chọn chức năng “Quản lý kết quả thi” từ menu chính.
	2	System	Hiển thị danh sách các kỳ thi với bộ lọc theo môn học, lớp, thời gian.
	3	User	Chọn kỳ thi cần xem và áp dụng các bộ lọc (tên học sinh, mã học sinh, mã đề, khoảng điểm).
	4	System	Hiển thị bảng kết quả với các cột: tên học sinh, mã đề, điểm, số câu đúng/sai, thời gian chấm.

Mục	Nội dung		
	5	User	Sắp xếp kết quả theo cột bất kỳ bằng cách nhấp vào tiêu đề cột.
	6	User	Nhấp vào một hàng để xem chi tiết kết quả của học sinh.
	7	System	Hiển thị chi tiết đáp án từng câu, hình ảnh phiếu đã chấm, lịch sử phúc tra (nếu có).
	8	User	Chỉnh sửa điểm nếu phát hiện sai sót và nhập lý do chỉnh sửa.
	9	User	Nhấn “Xuất Excel” để xuất kết quả ra file Excel với định dạng đã thiết lập.
	10	System	Tạo file Excel với đầy đủ thông tin và cung cấp link tải về.
	11	User	Nhấn “Tạo báo cáo tổng hợp” để tạo báo cáo theo mẫu có sẵn.
	12	System	Hiển thị báo cáo với biểu đồ phân bố điểm và thống kê tổng hợp.
Luồng thực thi mở rộng	No.	Thực hiện	Hành động
	3a	System	Thông báo: Không có kết quả nào khớp với bộ lọc, vui lòng điều chỉnh.
	8a	System	Thông báo lỗi: Vui lòng nhập lý do chỉnh sửa điểm.
	8b	System	Yêu cầu xác nhận trước khi lưu thay đổi điểm.
	9a	System	Thông báo lỗi: Không thể xuất file Excel, vui lòng thử lại.
	10a	System	Thông báo lỗi: Không thể tạo báo cáo tổng hợp, vui lòng liên hệ quản trị viên.
Điều kiện sau	Kết quả thi được hiển thị đầy đủ cho actor. Các chỉnh sửa điểm (nếu có) được lưu lại cùng lý do và lịch sử thay đổi. File Excel được tải về thành công để phục vụ báo cáo.		

Table 2.5: Đặc tả use case UC005 - Quản lý kết quả thi

2.2.6 Đặc tả use case Đăng nhập hệ thống

Mục	Nội dung		
Mã UC (UC #)	UC006		
Tên usecase	Đăng nhập hệ thống		
Tác nhân	Tất cả người dùng (Học sinh, Giáo viên, Quản trị trường, Quản trị hệ thống,		
Điều kiện trước	Người dùng đã có tài khoản được tạo trong hệ thống. Người dùng truy cập vào trang đăng nhập của ứng dụng web hoặc mobile. Kết nối mạng ổn định (hoặc đã có token hợp lệ được lưu cục bộ).		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Truy cập trang đăng nhập và nhập email và mật khẩu.
	2	User	Nhấn nút “Đăng nhập”.
	3	System	Validate định dạng email và kiểm tra các trường bắt buộc.
	4	System	Gửi yêu cầu xác thực đến server.
	5	System	Kiểm tra thông tin đăng nhập trong cơ sở dữ liệu.
	6	System	Sinh access token (JWT) và refresh token cho người dùng.
	7	System	Lưu refresh token an toàn và trả access token về client.
	8	System	Chuyển người dùng đến giao diện chính tương ứng với vai trò.
Luồng thực thi mở rộng	No.	Thực hiện	Hành động
	3a	System	Thông báo lỗi: Vui lòng nhập email và mật khẩu hợp lệ.
	3b	System	Thông báo lỗi: Định dạng email không đúng.
	5a	System	Thông báo lỗi: Email hoặc mật khẩu không chính xác.
	5b	System	Thông báo cảnh báo: Tài khoản đã bị khóa, vui lòng liên hệ quản trị viên.

Mục	Nội dung		
	5c	System	Thông báo cảnh báo: Quá nhiều lần đăng nhập thất bại, vui lòng thử lại sau 5 phút.
	7a	System	Thông báo lỗi: Không thể kết nối đến server, vui lòng kiểm tra kết nối mạng.
Điều kiện sau	Người dùng đã đăng nhập thành công và được chuyển đến giao diện chính phù hợp với vai trò. Access token được sử dụng cho các yêu cầu tiếp theo, refresh token được lưu trữ an toàn để làm mới token khi cần.		

Table 2.6: Đặc tả use case UC006 - Đăng nhập hệ thống

2.2.7 Đặc tả use case Tạo nhóm gia đình

Mục	Nội dung		
Mã UC (UC #)	UC007		
Tên usecase	Tạo nhóm gia đình		
Tác nhân	Người dùng (User - Phụ huynh)		
Điều kiện trước	Người dùng đã đăng nhập vào hệ thống.		
Luồng thực thi chính	No.	Thực hiện	Hành động
	1	User	Chọn chức năng “Tạo nhóm gia đình”.
	2	System	Hiển thị giao diện nhập thông tin nhóm (tên nhóm, mô tả, ảnh đại diện).
	3	User	Nhập tên nhóm và các thông tin cần thiết.
	4	System	Kiểm tra tính hợp lệ của dữ liệu (tên nhóm không trống).
	5	System	Lưu thông tin nhóm vào cơ sở dữ liệu.
	6	System	Hiển thị thông báo “Tạo nhóm gia đình thành công”.
	7	System	Chuyển người dùng đến giao diện “Quản lý nhóm”.
Luồng thực thi mở rộng	No.	Thực hiện	Hành động
	4a	System	Thông báo lỗi: Vui lòng nhập tên nhóm.
	5a	System	Thông báo lỗi: Không thể tạo nhóm. Vui lòng thử lại sau.

Mục	Nội dung
Điều kiện sau	Nhóm gia đình mới được tạo thành công và người dùng trở thành Trưởng nhóm. Hệ thống sẵn sàng cho việc thêm thành viên và chia sẻ danh sách.

Table 2.7: Đặc tả use case UC007 - Tạo nhóm gia đình

Bảng ?? dưới đây thể hiện ma trận truy vết giữa các use case đã đặc tả và các yêu cầu chức năng chính, giúp đảm bảo tất cả yêu cầu đều được triển khai thông qua các use case tương ứng.

2.3 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng về hiệu năng, tính khả dụng, bảo mật, và khả năng mở rộng như được trình bày trong Bảng 2.8.

Tiêu chí	Chỉ tiêu cụ thể	Giải thích chi tiết
Hiệu năng xử lý	Thời gian nhận diện OMR dưới 3 giây trên thiết bị tầm trung	Thiết bị tầm trung được định nghĩa là smartphone có RAM từ 4GB trở lên, chip xử lý từ thế hệ 2019 trở lại đây. Thời gian được đo từ khi chụp ảnh đến khi hiển thị kết quả hoàn chỉnh bao gồm hình ảnh đã đánh dấu và điểm số.
Độ chính xác nhận diện	Tỷ lệ nhận diện đúng tối thiểu 95%	Tỷ lệ được đo trên tập dữ liệu gồm 1000 phiếu thi với nhiều điều kiện ánh sáng và góc chụp khác nhau. Phiếu được đánh giá là nhận diện đúng nếu tất cả các câu hỏi đều được nhận diện đúng đáp án.
Độ trễ API	Thời gian phản hồi API dưới 500ms đối với 95% yêu cầu	Đo lường trên server có cấu hình 2 vCPU, 4GB RAM chạy trên nền tảng cloud với kết nối mạng ổn định. Loại trừ các API liên quan đến tải file hoặc gọi AI bên ngoài.

Tiêu chí	Chỉ tiêu cụ thể	Giải thích chi tiết
Khả năng chịu tải	Hỗ trợ 500 người dùng đồng thời	Người dùng đồng thời được định nghĩa là người dùng đang thực hiện thao tác trong khoảng thời gian 10 giây. Hệ thống cần duy trì thời gian phản hồi dưới 2 giây đối với tất cả API.
Tính khả dụng	Uptime 99.5% đối với backend	Cho phép downtime tối đa 3.6 giờ mỗi tháng cho mục đích bảo trì. Thời gian bảo trì được lên kế hoạch trước và thông báo cho người dùng ít nhất 24 giờ.
Tính di động	Hỗ trợ iOS 13+ và Android 8+	Đảm bảo ứng dụng chạy mượt trên ít nhất 95% thiết bị đang hoạt động trên thị trường Việt Nam tính đến năm 2025.
Bảo mật	Dữ liệu được mã hóa khi truyền qua mạng (TLS 1.3)	Tất cả API endpoints phải sử dụng HTTPS với chứng chỉ hợp lệ [8]. Token JWT [9] có thời hạn tối đa 24 giờ. Refresh token được lưu trữ an toàn trong keychain (iOS) hoặc keystore (Android).
Quyền riêng tư	Tuân thủ quy định về bảo vệ dữ liệu cá nhân	Dữ liệu cá nhân của học sinh (họ tên, mã học sinh, kết quả thi) chỉ được truy cập bởi giáo viên và phụ huynh có liên kết hợp lệ. Không sử dụng dữ liệu cho mục đích thương mại. Cho phép người dùng yêu cầu xóa dữ liệu.

Tiêu chí	Chỉ tiêu cụ thể	Giải thích chi tiết
Khả năng mở rộng	Kiến trúc modular cho phép scale từng module	Backend được thiết kế theo mô hình modular, cho phép nhân bản các module có tải cao như API xác thực hoặc API nhận diện OMR mà không ảnh hưởng đến các module khác.
Dễ sử dụng	Người dùng mới có thể thực hiện thao tác quét OMR sau dưới 5 phút hướng dẫn	Được kiểm chứng qua bài kiểm tra với 30 người dùng chưa từng sử dụng ứng dụng. Thời gian hoàn thành thao tác quét đầu tiên được đo từ khi mở ứng dụng đến khi nhận được kết quả chấm.

Table 2.8: Các yêu cầu phi chức năng của hệ thống

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

3.1 Công nghệ xử lý ảnh OMR

Optical Mark Recognition (OMR) là công nghệ cho phép nhận diện các vạch hoặc ô được đánh dấu trên giấy thông qua việc phân tích hình ảnh. Khác với OCR (Optical Character Recognition) nhằm nhận diện ký tự văn bản [10], OMR tập trung vào việc xác định xem một vùng tròn hoặc hình vuông có được tô đen hay không, thông qua việc đo mật độ pixel trong vùng đó. Đây là công nghệ cốt lõi giúp hệ thống SMART GRADING tự động chấm thi trắc nghiệm mà không cần con người đọc và đánh dấu từng câu.

Nguyên lý hoạt động của OMR bao gồm ba giai đoạn chính dựa trên các kỹ thuật xử lý ảnh kinh điển đã được Gonzalez và Woods tổng hợp trong tài liệu [11], đồng thời được hỗ trợ bởi thư viện OpenCV [6]. Trong giai đoạn thứ nhất là tiền xử lý ảnh, hệ thống đọc ảnh đầu vào từ camera hoặc file hình ảnh, chuyển đổi ảnh màu sang ảnh xám (grayscale) để giảm độ phức tạp của dữ liệu, cân bằng histogram để cải thiện độ tương phản giữa vùng được tô và vùng trắng, áp dụng bộ lọc Gaussian để giảm nhiễu hạt, và phát hiện các cạnh trong ảnh bằng thuật toán Canny [12], [13]. Trong giai đoạn thứ hai là phát hiện vùng phiếu, hệ thống tìm các contour trong ảnh đã qua xử lý bằng thuật toán border following [14] để xác định hình dạng tổng thể của phiếu, áp dụng phép biến đổi Hough để phát hiện các đường thẳng tạo thành cạnh phiếu, và tính toán góc nghiêng để chuẩn bị cho bước hiệu chỉnh phối cảnh. Trong giai đoạn thứ ba là nhận diện bubble, hệ thống sử dụng template JSON chứa tọa độ chính xác của từng bubble trên phiếu để trích xuất vùng ảnh tương ứng với mỗi bubble, tính toán mật độ pixel (intensity) trong vùng bubble bằng công thức lấy trung bình giá trị grayscale của tất cả pixel trong vùng, và so sánh với ngưỡng (threshold) để quyết định bubble được tô (intensity thấp, gần màu đen) hay chưa tô (intensity cao, gần màu trắng).

Điểm then chốt quyết định độ chính xác của OMR nằm ở hai yếu tố. Thứ nhất là chất lượng ảnh đầu vào, bao gồm độ phân giải (DPI), điều kiện ánh sáng khi chụp, và góc chụp của camera. Đối với ảnh từ camera smartphone với độ phân giải 12 megapixel trở lên, chất lượng thường đủ tốt để nhận diện chính xác. Thứ hai là ngưỡng intensity được sử dụng để phân loại bubble tô và chưa tô. Trong hệ thống SMART GRADING, ngưỡng này được thiết lập mặc định là 180 (trên thang 0-255, trong đó 0 là đen hoàn toàn và 255 là trắng hoàn toàn), có thể điều chỉnh được tùy theo loại giấy và bút sử dụng.

So với các phương pháp nhận diện khác như sử dụng mạng nơ-ron tích chập

(CNN) để nhận diện bubble [15], [16], OMR truyền thống dựa trên mật độ pixel có ưu điểm vượt trội về tốc độ xử lý và không yêu cầu dữ liệu huấn luyện. Với một phiếu có 50 câu hỏi và 4 lựa chọn mỗi câu, OMR truyền thống chỉ cần xử lý 200 vùng ảnh nhỏ thay vì phân tích toàn bộ ảnh bằng CNN, giúp giảm đáng kể thời gian xử lý từ hàng chục giây xuống còn dưới 2 giây trên thiết bị di động. Đây là lý do hệ thống SMART GRADING lựa chọn phương pháp OMR dựa trên mật độ pixel kết hợp với template JSON chứa tọa độ chính xác, thay vì sử dụng mô hình học sâu đòi hỏi nhiều tài nguyên tính toán.

[Hình minh họa: Lưu đồ quy trình xử lý OMR]

Figure 3.1: Lưu đồ quy trình xử lý OMR trong hệ thống SMART GRADING

Hình 3.1 minh họa lưu đồ quy trình xử lý OMR từ khi chụp ảnh đến khi có kết quả chấm điểm. Điểm đặc biệt trong thiết kế của hệ thống SMART GRADING là toàn bộ quy trình xử lý OMR được thực hiện trên thiết bị di động (client-side), không cần gửi ảnh lên server. Điều này mang lại ba lợi ích quan trọng gồm thời gian phản hồi tức thì cho người dùng, hoạt động được trong môi trường không có internet, và dữ liệu phiếu thi không rời khỏi thiết bị của giáo viên, đảm bảo tính riêng tư.

3.2 Nền tảng phát triển ứng dụng di động Flutter

Flutter là một framework cross-platform được phát triển bởi Google, cho phép xây dựng ứng dụng native cho iOS và Android từ một codebase duy nhất bằng ngôn ngữ lập trình Dart. Được công bố lần đầu vào năm 2017 và phiên bản ổn định đầu tiên ra mắt vào tháng 12 năm 2018, Flutter đã nhanh chóng trở thành một trong những framework phát triển di động phổ biến nhất thế giới với hàng trăm nghìn ứng dụng trên cả hai nền tảng iOS và Android.

Kiến trúc của Flutter được thiết kế theo mô hình ba lớp rõ ràng. Lớp dưới cùng là engine Skia được viết bằng C++, có nhiệm vụ vẽ trực tiếp lên canvas của thiết bị mà không thông qua bất kỳ lớp trung gian nào như WebView hoặc bridge. Đây là điểm khác biệt quan trọng so với các framework cross-platform khác như React Native, vốn sử dụng bridge để giao tiếp với các component native. Lớp ở giữa là các binding (Dart FFI) kết nối engine với mã Dart. Lớp trên cùng là framework Dart chứa tất cả các widget, class và function mà lập trình viên sử dụng để xây dựng ứng dụng. Nhờ kiến trúc này, Flutter đạt được hiệu suất gần như ngang hàng

với ứng dụng native, với khả năng đạt 60fps hoặc thậm chí 120fps trên các thiết bị có màn hình hỗ trợ.

Trong hệ thống SMART GRADING, Flutter được sử dụng để xây dựng toàn bộ ứng dụng di động bao gồm module chụp ảnh và nhận diện OMR, module xem kết quả và lịch sử thi, module thông báo và nhắc nhở, và module quản lý tài khoản. Việc lựa chọn Flutter thay vì phát triển ứng dụng native cho từng nền tảng iOS (Swift/UIKit) và Android (Kotlin/Android SDK) mang lại lợi ích đáng kể về mặt chi phí phát triển và bảo trì. Theo ước tính, việc phát triển và bảo trì một ứng dụng cross-platform bằng Flutter tiết kiệm khoảng 40-50% thời gian và chi phí so với việc phát triển hai ứng dụng native riêng biệt.

Để quản lý trạng thái ứng dụng phức tạp, hệ thống sử dụng mẫu thiết kế BLoC (Business Logic Component) kết hợp với thư viện flutter_bloc. Mẫu BLoC tách biệt hoàn toàn phân giao diện người dùng (presentation layer) khỏi logic nghiệp vụ (business logic layer) thông qua các stream và event. Khi người dùng tương tác với giao diện, một event được phát sinh và gửi đến BLoC tương ứng. BLoC xử lý event và phát sinh state mới, giao diện lắng nghe state và tự động cập nhật khi state thay đổi. Cách thiết kế này giúp mã nguồn dễ kiểm thử hơn, dễ tái sử dụng hơn, và đặc biệt là các thành phần BLoC có thể được tái sử dụng ở cả ứng dụng mobile và web nếu cần.

Một thư viện quan trọng khác được sử dụng là GetIt, một service locator cho phép dependency injection một cách đơn giản và hiệu quả. GetIt được sử dụng để đăng ký và truy xuất các instance của service và repository trên toàn bộ ứng dụng, thay thế cho cách truyền tham số qua constructor lồng nhau (prop drilling) vốn khó quản lý trong ứng dụng lớn. Các service như ApiClient, ExamService, SubmissionService, và OmrEngineService đều được đăng ký với GetIt và có thể truy xuất từ bất kỳ đâu trong ứng dụng.

3.3 Nền tảng phát triển ứng dụng web React

React là một thư viện JavaScript được phát triển và duy trì bởi Meta (trước đây là Facebook), ra mắt lần đầu vào năm 2013. React cho phép xây dựng giao diện người dùng thông qua mô hình component-based, trong đó mỗi phần giao diện được tách thành các component độc lập có thể quản lý trạng thái (state) và vòng đời (lifecycle) riêng. Điểm nổi bật nhất của React là Virtual DOM, một bản sao của DOM thực trong bộ nhớ, cho phép React tính toán sự khác biệt (diffing) giữa DOM hiện tại và DOM mới một cách hiệu quả, từ đó chỉ cập nhật những phần thực sự thay đổi thay vì re-render toàn bộ trang.

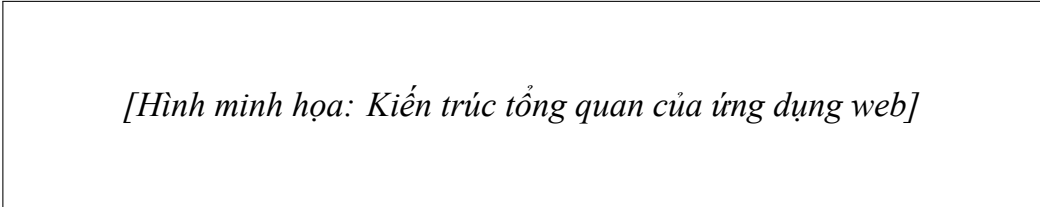
React 19, phiên bản mới nhất được phát hành vào cuối năm 2024, mang đến

nhiều cải tiến đáng kể bao gồm compiler mới (React Compiler) tự động tối ưu hóa mã, hỗ trợ native form handling, server components được cải thiện, và các API mới cho phép quản lý side effect tốt hơn. Trong hệ thống SMART GRADING, ứng dụng web được phát triển dựa trên React 19 để tận dụng những cải tiến này, đặc biệt là server components giúp giảm JavaScript được tải về phía client và cải thiện thời gian khởi động ban đầu.

TypeScript, một ngôn ngữ lập trình là superset của JavaScript với hệ thống kiểu tĩnh, được sử dụng kết hợp với React trong toàn bộ dự án web. TypeScript giúp phát hiện lỗi type ngay trong quá trình phát triển (compile time) thay vì để lỗi phát sinh khi chạy (runtime), đặc biệt quan trọng trong các dự án lớn với nhiều module và nhiều lập trình viên cùng tham gia. Ngoài ra, TypeScript cung cấp tính năng auto-complete và refactoring support mạnh mẽ trong các IDE, giúp tăng năng suất phát triển đáng kể.

Công cụ build Vite được lựa chọn thay cho webpack truyền thống nhờ tốc độ vượt trội. Vite sử dụng ES modules gốc của trình duyệt trong chế độ phát triển (dev mode), cho phép khởi động server gần như tức thì với thời gian dưới một giây ngay cả với dự án có quy mô lớn. Trong chế độ production, Vite sử dụng Rollup để bundle mã nguồn, tạo ra các bundle có kích thước nhỏ gọn và được chia nhỏ (code splitting) một cách thông minh.

Về quản lý trạng thái, ứng dụng web sử dụng Zustand, một thư viện quản lý state nhẹ với API rất đơn giản. So với Redux, Zustand giảm đáng kể boilerplate code (mã lặp lại) mà không hy sinh tính năng cần thiết. Một store Zustand có thể được tạo chỉ với vài dòng code và sử dụng trực tiếp trong component mà không cần Provider wrapper như Redux. Đối với việc quản lý server state (dữ liệu được fetch từ API), hệ thống sử dụng TanStack Query (trước đây là React Query), thư viện giúp quản lý caching, refetching, và đồng bộ hóa dữ liệu server một cách hiệu quả, giảm đáng kể code cần viết so với việc sử dụng useEffect và state thông thường.



[Hình minh họa: Kiến trúc tổng quan của ứng dụng web]

Figure 3.2: Kiến trúc tổng quan của ứng dụng web SMART GRADING

Hình 3.2 thể hiện kiến trúc tổng quan của ứng dụng web, trong đó các component React được tổ chức theo cấu trúc phân lớp từ giao diện người dùng (presentation)

đến quản lý trạng thái (state management) và cuối cùng là giao tiếp với backend qua các service layer.

3.4 Kiến trúc backend Node.js và Express

Node.js là một runtime JavaScript được xây dựng trên engine V8 của Chrome, cho phép thực thi mã JavaScript phía server. Điểm mạnh nổi bật nhất của Node.js là mô hình I/O non-blocking và event-driven, cho phép xử lý đồng thời hàng nghìn kết nối mà không cần tạo thread mới cho mỗi kết nối như các web server truyền thống. Điều này đặc biệt phù hợp với các ứng dụng web hiện đại vốn đặc trưng bởi nhiều thao tác I/O như đọc ghi database, gọi API bên ngoài, và xử lý file upload.

Express là framework web phổ biến nhất cho Node.js, cung cấp một lớp trừu tượng mỏng và linh hoạt trên nền tảng Node.js để xây dựng API và ứng dụng web. Express không áp đặt kiến trúc cứng nhắc, cho phép lập trình viên tự do thiết kế cấu trúc ứng dụng phù hợp với yêu cầu cụ thể. Trong hệ thống SMART GRADING, backend được tổ chức theo kiến trúc phân lớp MVC (Model-View-Controller) với ba lớp chính gồm routes tiếp nhận và định tuyến request, controllers xử lý logic điều khiển và validation dữ liệu đầu vào, và services chứa logic nghiệp vụ chính và tương tác với database.

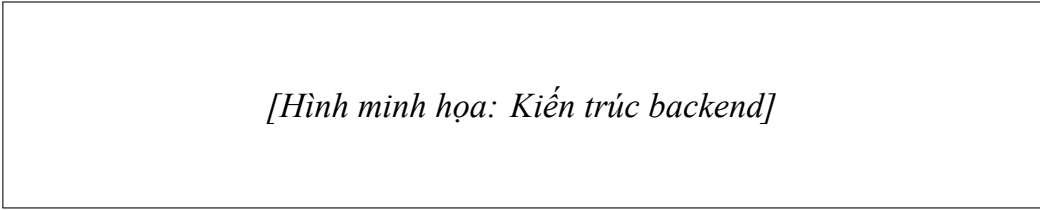
MongoDB, một cơ sở dữ liệu NoSQL document-oriented, được lựa chọn làm nơi lưu trữ dữ liệu chính cho hệ thống. So sánh giữa các hệ cơ sở dữ liệu NoSQL và quan hệ đã được Han và cộng sự khảo sát [17]. Khác với cơ sở dữ liệu quan hệ truyền thống (SQL) yêu cầu định nghĩa schema cứng nhắc trước khi lưu dữ liệu, MongoDB cho phép lưu trữ dữ liệu dưới dạng document JSON linh hoạt, rất phù hợp với cấu trúc dữ liệu phức tạp và thay đổi thường xuyên như cấu trúc câu hỏi thi, đáp án, và kết quả chấm, đồng thời đáp ứng được yêu cầu về tính nhất quán và khả dụng theo định lý CAP [18]. Trong hệ thống SMART GRADING, một document exam có thể chứa danh sách questions với mỗi question có cấu trúc riêng bao gồm nội dung, các phương án, và metadata, thay vì phải tách thành nhiều bảng liên kết phức tạp như trong SQL.

Mongoose được sử dụng như ODM (Object Data Modeling) để định nghĩa schema và tương tác với MongoDB. Mongoose cung cấp các tính năng quan trọng bao gồm schema validation tự động khi lưu document, middleware (hook) cho phép chạy logic trước và sau các thao tác database, populate cho phép tự động điền dữ liệu từ collection liên kết, và các phương thức tiện ích như findOneAndUpdate hay aggregate giúp thực hiện các truy vấn phức tạp một cách dễ dàng.

Về xác thực và phân quyền, hệ thống sử dụng JWT (JSON Web Token) [9] kết hợp với mô hình RBAC (Role-Based Access Control). Khi người dùng đăng nhập

thành công, server tạo một JWT chứa thông tin về user ID, role, và thời hạn hiệu lực, sau đó gửi về cho client. Client lưu trữ token này và gửi kèm trong header của mọi request tiếp theo để xác thực. Cơ chế ủy quyền OAuth 2.0 [19] cũng được tích hợp cho phép tích hợp với các nhà cung cấp danh tính bên thứ ba. Middleware xác thực trên server giải mã token và kiểm tra quyền truy cập dựa trên vai trò của người dùng. JWT được ưa chuộng nhờ tính stateless, không cần server lưu trữ session, và có thể được xác minh bởi bất kỳ server nào có khóa bí mật.

Joi được sử dụng cho việc validate dữ liệu đầu vào ở tầng controller trước khi chuyển xuống service. Joi cho phép định nghĩa schema validation với cú pháp rõ ràng và dễ đọc, hỗ trợ nhiều loại validation phong phú, và tự động sinh message lỗi chi tiết khi validation thất bại. Winston và Morgan được sử dụng cho logging, trong đó Winston xử lý ghi log ra file và console với nhiều mức độ (error, warn, info, debug), còn Morgan là middleware ghi lại HTTP request với thông tin về method, URL, status code, và thời gian response.



[Hình minh họa: Kiến trúc backend]

Figure 3.3: Kiến trúc backend của hệ thống SMART GRADING

Hình 3.3 thể hiện kiến trúc backend tổng thể với các thành phần được tổ chức theo kiến trúc phân lớp rõ ràng, từ API Gateway (middleware) đến Business Logic (services) và Data Access (Mongoose models), cùng với các thành phần bên ngoài là MongoDB, Cloudinary và các AI API.

3.5 Công cụ AMC và tích hợp hệ thống

Auto Multiple Choice (AMC) là phần mềm mã nguồn mở được phát triển bằng Perl và LaTeX, cho phép tạo ra các kỳ thi trắc nghiệm với nhiều mã đề một cách tự động. AMC hỗ trợ ba phương thức sinh mã đề chính gồm xáo trộn thứ tự câu hỏi trong cùng một đề gốc, xáo trộn thứ tự các phương án trong từng câu hỏi, và kết hợp cả hai phương thức trên. Kết quả đầu ra bao gồm các file PDF chứa đề thi cho từng mã đề và các file dữ liệu (‘.csv’, ‘.sqlite’) chứa đáp án tương ứng với từng mã đề.

Điểm then chốt giúp AMC phù hợp với hệ thống SMART GRADING là khả năng xuất tọa độ OMR. Khi compile đề thi, AMC tự động sinh ra file ‘data.calage.xy’ chứa tọa độ pixel chính xác của từng bubble trên phiếu trả lời, được tính toán dựa

trên cấu hình trang giấy (A4, margins), cỡ chữ, và layout của phiếu trong mã nguồn LaTeX. File này chứa thông tin về vị trí (x, y) và kích thước (width, height) của mỗi bubble, cùng với mã số câu hỏi và phương án tương ứng. Hệ thống SMART GRADING đọc file này và chuyển đổi thành template JSON, sử dụng trực tiếp cho engine nhận diện OMR trên mobile mà không cần bất kỳ phép tính hay hiệu chỉnh thủ công nào.

Quy trình tích hợp AMC vào hệ thống SMART GRADING hoạt động như sau. Đầu tiên, giáo viên tải file LaTeX chứa đề thi gốc lên ứng dụng web. File LaTeX này được viết theo cú pháp AMC, trong đó mỗi câu hỏi được định nghĩa bằng môi trường ‘question’ và các phương án được định nghĩa bằng ‘choices’. Sau khi upload, server gọi AMC CLI để compile file LaTeX, AMC sinh N file PDF (mỗi file cho một mã đề) và file ‘data.calage.xy’ chứa tọa độ OMR. Server đọc file tọa độ, chuyển đổi thành template JSON theo định dạng chuẩn của SMART GRADING, và lưu vào database cùng với các thông tin về exam ID, version code, và answer key. Các file PDF được lưu trữ trên Clouinary để giáo viên có thể tải về in ấn.

Lý do lựa chọn AMC thay vì tự xây dựng công cụ sinh đề là ba yếu tố chính. Thứ nhất, AMC là công cụ đã được kiểm chứng và sử dụng rộng rãi trong cộng đồng giáo dục nhiều năm, với hàng triệu đề thi đã được sinh ra. Thứ hai, AMC hỗ trợ đầy đủ các tính năng cần thiết bao gồm xáo trộn câu hỏi, xáo trộn đáp án, chia nhóm câu hỏi, và thiết lập trọng số điểm. Thứ ba, AMC là mã nguồn mở hoàn toàn miễn phí, không phát sinh chi phí license khi triển khai trên production.

[Hình minh họa: Lưu đồ tích hợp AMC]

Figure 3.4: Lưu đồ tích hợp AMC trong hệ thống SMART GRADING

Hình 3.4 minh họa lưu đồ tích hợp AMC, cho thấy cách file LaTeX được upload, compile bởi AMC CLI, và kết quả đầu ra bao gồm PDF đề thi và template JSON OMR được xử lý và lưu trữ trong hệ thống.

3.6 Các công cụ và dịch vụ hỗ trợ

Bên cạnh các nền tảng công nghệ chính, hệ thống SMART GRADING sử dụng một số công cụ và dịch vụ hỗ trợ để đảm bảo chất lượng và hiệu quả vận hành. Clouinary là dịch vụ đám mây chuyên về quản lý hình ảnh và video, được sử dụng trong hệ thống để lưu trữ và phân phối các file hình ảnh bao gồm ảnh gốc

phiếu thi, ảnh đã được ghi chú (annotated image) sau khi quét, và ảnh preview của đề thi. Cloudinary cung cấp các API tiện lợi cho việc upload, transform (resize, crop, watermark), và CDN để phân phối nhanh chóng đến người dùng cuối. Với gói miễn phí cho phép lưu trữ 25GB và bandwidth 25GB mỗi tháng, Cloudinary đủ đáp ứng cho giai đoạn phát triển và thử nghiệm.

Docker được sử dụng để đóng gói và triển khai backend server, đảm bảo tính nhất quán giữa môi trường phát triển và môi trường production. Docker container chứa toàn bộ runtime environment bao gồm Node.js, các npm dependencies, và AMC CLI, giúp quá trình deploy trở nên đơn giản và an toàn. Docker Compose được sử dụng để định nghĩa và chạy multi-container application, bao gồm container cho backend server, container cho MongoDB, và container cho Redis (cache layer). PM2 được sử dụng như process manager cho Node.js application trong production, cung cấp tính năng auto-restart khi process crash, load balancing giữa các process, và log management.

GitHub Actions được sử dụng cho CI/CD pipeline, tự động chạy các bài kiểm thử (unit test, integration test) mỗi khi có commit mới, và tự động deploy lên server staging khi merge vào nhánh develop. GitHub Actions cũng được cấu hình để chạy linting và code quality check trước khi cho phép merge, đảm bảo chất lượng mã nguồn trước khi đưa vào codebase chính.

Google Gemini API là nền tảng AI được tích hợp để cung cấp các tính năng trí tuệ nhân tạo bao gồm sinh câu hỏi tự động, phân tích kết quả học tập, và chatbot trả lời câu hỏi. Gemini được lựa chọn nhờ khả năng xử lý ngữ cảnh dài với context window lên đến 1 triệu token [20], cho phép phân tích toàn diện toàn bộ kết quả thi của một học sinh trong một lần gọi API. Kiến trúc transformer đa modal của Gemini được xây dựng dựa trên nền tảng attention mechanism [21] và kế thừa các cải tiến từ các mô hình ngôn ngữ lớn trước đó như BERT [22]. So với các lựa chọn thay thế như OpenAI GPT-4 hay Anthropic Claude, Gemini có mức giá cạnh tranh hơn và tích hợp tốt với hệ sinh thái Google Cloud Platform.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Tổng quan kiến trúc hệ thống

Hệ thống SMART GRADING được xây dựng theo kiến trúc phân tán ba lớp (three-tier distributed architecture) [23], trong đó mỗi lớp đảm nhận một nhiệm vụ riêng biệt và giao tiếp với các lớp khác thông qua các giao thức đã được chuẩn hóa. Kiến trúc này cho phép mở rộng theo chiều ngang (horizontal scaling) khi lượng người dùng tăng lên, đồng thời đảm bảo tính độc lập giữa các thành phần để dễ dàng bảo trì và nâng cấp. Hệ thống bao gồm ba thành phần chính tương ứng với ba lớp hoặc ba platform riêng biệt, mỗi thành phần được phát triển trên nền tảng công nghệ phù hợp nhất với yêu cầu và đặc điểm sử dụng của nó.

[Hình minh họa: Kiến trúc tổng quan hệ thống]

Figure 4.1: Kiến trúc tổng quan hệ thống SMART GRADING với ba thành phần chính

Hình 4.1 thể hiện kiến trúc tổng quan của toàn bộ hệ thống SMART GRADING, trong đó phần bên trái là ứng dụng di động Flutter, phần trung tâm là backend Node.js/Express, phần bên phải là ứng dụng web React, và phần dưới là các thành phần dữ liệu và dịch vụ bên ngoài. Các mũi tên biểu diễn hướng đi của dữ liệu và lời gọi API. Điểm đặc biệt trong kiến trúc này là sự phân chia rõ ràng giữa hai luồng xử lý chính gồm luồng AMC Exam (mobile-first với OMR chạy trên thiết bị) và luồng Legacy Exam (backend-side với Python bridge xử lý OMR trên server), cho phép hệ thống linh hoạt hỗ trợ nhiều loại đề thi khác nhau.

Lớp thứ nhất là lớp trình bày (Presentation Layer) bao gồm hai ứng dụng client được phát triển trên hai nền tảng khác nhau. Ứng dụng di động SMART GRADING Mobile được xây dựng bằng Flutter phiên bản 3.22 và ngôn ngữ Dart phiên bản 3.4, cho phép cài đặt trên cả hệ điều hành iOS (từ iOS 13 trở lên) và Android (từ Android 8.0 trở lên). Điểm nổi bật của ứng dụng mobile là engine nhận diện OMR (OMR Engine v2) được triển khai hoàn toàn trên thiết bị client (edge computing), cho phép xử lý ảnh và chấm điểm ngay lập tức mà không cần gửi dữ liệu lên server. Ứng dụng web SMART GRADING Web được xây dựng bằng React phiên bản 19 và

TypeScript phiên bản 5.5, sử dụng công cụ build Vite phiên bản 5 để đảm bảo tốc độ phát triển và production build nhanh chóng. Ứng dụng web cung cấp giao diện quản lý cho giáo viên và quản trị viên, cho phép tạo đề thi, quản lý kết quả, xuất báo cáo, và cấu hình hệ thống.

Lớp thứ hai là lớp ứng dụng (Application Layer) là backend server được phát triển bằng Node.js phiên bản 20 LTS và framework Express phiên bản 4.21. Backend server đóng vai trò là API Gateway, tiếp nhận và định tuyến tất cả các request từ các ứng dụng client, thực thi logic nghiệp vụ, và tương tác với cơ sở dữ liệu cũng như các dịch vụ bên ngoài. Thiết kế RESTful API tuân theo các nguyên tắc đã được Richardson và cộng sự trình bày [24]. Backend được thiết kế theo kiểu monolith modular, trong đó các module (auth, exam, submission, ai) được tổ chức trong các thư mục riêng biệt nhưng triển khai trong cùng một process Node.js, đơn giản hóa quá trình phát triển trong giai đoạn đầu trong khi vẫn duy trì sự tách biệt về mặt logic giữa các module. Kiến trúc này thuận tiện cho việc tách ra thành microservices trong tương lai khi hệ thống mở rộng.

Lớp thứ ba là lớp dữ liệu (Data Layer) bao gồm ba thành phần lưu trữ chính. MongoDB phiên bản 7.0 đóng vai trò là cơ sở dữ liệu chính, lưu trữ toàn bộ dữ liệu cấu trúc của hệ thống bao gồm thông tin người dùng, đề thi, kết quả, và cấu hình OMR. Cloudinary là dịch vụ lưu trữ file đám mây chuyên về hình ảnh, được sử dụng để lưu trữ các file đề thi PDF, ảnh gốc phiếu thi, và ảnh đã được ghi chú sau khi quét. Redis là hệ thống cache in-memory được sử dụng để tăng tốc các truy vấn thường xuyên và giảm tải cho MongoDB.

Điểm kiến trúc quan trọng nhất và cũng là điểm khác biệt cốt lõi so với các giải pháp OMR truyền thống là việc engine nhận diện OMR được triển khai trên thiết bị di động (client-side) thay vì trên server (server-side). Thiết kế này, thường được gọi là edge computing hoặc fog computing, mang lại ba lợi ích quan trọng. Thứ nhất, thời gian phản hồi tức thì không phụ thuộc vào độ trễ mạng, với kết quả được hiển thị ngay sau khi chụp ảnh mà không cần chờ round-trip đến server. Thứ hai, hệ thống hoạt động được hoàn toàn offline trong môi trường không có internet, đặc biệt quan trọng đối với các trường học tại vùng nông thôn hoặc khu vực có hạ tầng mạng không ổn định. Thứ ba, dữ liệu phiếu thi không rời khỏi thiết bị của giáo viên, đảm bảo tính riêng tư và bảo mật dữ liệu. Template JSON chứa tọa độ OMR được đồng bộ từ server về thiết bị khi bắt đầu phiên quét, sau đó mọi tính toán nhận diện đều thực hiện cục bộ trên thiết bị.

Hệ thống hỗ trợ hai luồng chấm điểm song song được quyết định bởi trường exam.paperEngine trong database. Luồng AMC Exam (mới) sử dụng AMC CLI

để sinh đề thi, mobile OMR engine (engine_v2) để quét OMR trên thiết bị di động, và templateJson từ OMRTemplate.templateJson để nhận diện. Luồng Legacy Exam (pdfkit) sử dụng pdfkit để sinh đề thi, Python bridge (OMRChecker) để quét OMR trên server, và zones từ OMRTemplate.zones cho cấu hình vị trí. Việc phân tách rõ ràng này cho phép hệ thống tương thích ngược với các đề thi được tạo bằng phương pháp cũ đồng thời hỗ trợ luồng mới với độ chính xác cao hơn.

Bảng 4.1 dưới đây tổng hợp các công cụ và môi trường phát triển đã sử dụng trong toàn bộ hệ thống.

Thành phần	Công cụ	Phiên bản
Ngôn ngữ lập trình (Mobile)	Dart	3.4.x
Framework (Mobile)	Flutter	3.22.x
State management (Mobile)	flutter_bloc	8.x
Dependency injection (Mobile)	GetIt	7.x
HTTP client (Mobile)	Dio	5.x
Local storage (Mobile)	SharedPreferences, SQLite	-
Ngôn ngữ lập trình (Web)	TypeScript	5.5.x
Framework (Web)	React	19.x
Build tool (Web)	Vite	5.x
State management (Web)	Zustand	4.x
Server state (Web)	TanStack Query	5.x
Routing (Web)	React Router	6.x
Ngôn ngữ lập trình (Backend)	JavaScript (ES2024)	-
Runtime (Backend)	Node.js	20.x LTS
Framework (Backend)	Express.js	4.21.x
Database	MongoDB	7.0.x
ODM	Mongoose	8.x
Validation (Backend)	Joi	17.x
Logging (Backend)	Winston	3.x

Thành phần	Công cụ	Phiên bản
Authentication	Passport.js + JWT	-
Image storage	Cloudinary	-
Cache	Redis	7.x
Container	Docker	26.x
Process Manager	PM2	5.x
Version Control	Git	2.45.x
CI/CD	GitHub Actions	-
LaTeX Compiler	TeX Live / AMC	2024
AI API	Google Gemini	1.5 Flash
API Documentation	Swagger	-

Table 4.1: Bảng tổng hợp công cụ phát triển

4.2 Thiết kế chi tiết

Phần này trình bày chi tiết việc thiết kế cơ sở dữ liệu MongoDB, thiết kế giao diện người dùng, và thiết kế lớp của hệ thống SMART GRADING.

4.2.1 Thiết kế cơ sở dữ liệu

Phần này trình bày chi tiết việc thiết kế cơ sở dữ liệu MongoDB của hệ thống SMART GRADING, bao gồm lý do lựa chọn NoSQL, cấu trúc các collection chính, các trường được đánh index để tối ưu hiệu năng truy vấn, và chiến lược lưu trữ dữ liệu.

Lý do lựa chọn MongoDB thay vì cơ sở dữ liệu quan hệ (SQL) cho hệ thống SMART GRADING dựa trên bốn yếu tố chính. Thứ nhất, MongoDB là cơ sở dữ liệu document-oriented lưu trữ dữ liệu dưới dạng BSON (Binary JSON), rất phù hợp với cấu trúc dữ liệu phức tạp và thay đổi thường xuyên như cấu trúc câu hỏi trong ngân hàng đề thi (mỗi câu hỏi có thể có số lượng phương án khác nhau, có thể chứa công thức LaTeX, hình ảnh, và metadata đa dạng), cấu trúc đáp án trong kết quả thi (cần lưu chi tiết từng câu với nhiều trường metadata), và cấu trúc template OMR với tọa độ không gian phức tạp. Thứ hai, MongoDB hỗ trợ nested documents cho phép lưu trữ các cấu trúc phân cấp tự nhiên như câu hỏi lồng nhau trong đề thi, đáp án lồng nhau trong submission, mà không cần phải tách thành nhiều bảng liên kết phức tạp như trong SQL. Thứ ba, MongoDB có khả năng mở rộng ngang (horizontal scaling) thông qua sharding, cho phép hệ thống xử lý lượng dữ liệu lớn khi số lượng người dùng và kỳ thi tăng lên mà không cần thay đổi kiến trúc. Thứ tư, MongoDB là mã nguồn mở với cộng đồng sử dụng rộng lớn và chi phí vận hành thấp khi sử dụng MongoDB Atlas cloud service hoặc tự host trên server riêng.

Cơ sở dữ liệu SMART GRADING bao gồm các collection chính sau đây. Collection User lưu trữ thông tin về người dùng hệ thống với cấu trúc document như sau. Trường email là duy nhất trên toàn bộ hệ thống và được đánh index để phục vụ truy vấn đăng nhập. Trường passwordHash lưu mật khẩu đã được băm bằng thuật toán bcrypt với salt rounds bằng 12, đảm bảo bảo mật ngay cả khi database bị lộ. Trường role là enum với năm giá trị gồm admin cho quản trị viên hệ thống có toàn quyền, school-admin cho quản trị viên trường có quyền quản lý trong phạm vi trường mình, teacher cho giáo viên có quyền tạo đề và xem kết quả, student cho học sinh có quyền xem điểm và nộp phúc tra, và parent cho phụ huynh có quyền theo dõi kết quả học tập của con em. Trường schoolId và classId tham chiếu đến các collection tương ứng. Trường linkedParentIds là mảng chứa các user ID của phụ huynh được liên kết, cho phép một học sinh có thể được theo dõi bởi nhiều phụ huynh.

Collection Exam là thực thể trung tâm của hệ thống, lưu trữ thông tin về kỳ thi với vai trò quan trọng trong việc điều phối luồng chấm điểm. Trường paperEngine là trường quyết định then chốt, là enum với hai giá trị gồm amc cho luồng AMC mới sử dụng mobile OMR engine, và pdfkit hoặc legacy cho luồng cũ sử dụng Python bridge. Trường status là enum với năm giá trị gồm draft khi đề thi đang được soạn thảo, published khi đề thi đã được công bố cho học sinh, in-progress khi đang trong thời gian thi, completed khi kỳ thi đã kết thúc, và archived khi đề thi được lưu trữ. Trường settings là embedded document chứa các cấu hình bao gồm allowLateSubmission cho phép nộp muộn, showResultToStudent cho phép học sinh xem kết quả, requiresRecheck yêu cầu giáo viên xác nhận trước khi công bố, allowMultipleAttempts cho phép thi nhiều lần, và shuffleOptions xáo trộn thứ tự phương án khi hiển thị.

Collection OMRTemplate là KEY COLLECTION lưu trữ cấu hình OMR với cấu trúc đặc biệt bao gồm hai phần chính. Phần thứ nhất là trường templateJson là embedded document chứa cấu hình OMR theo định dạng AMC, đây là single source of truth cho mobile OMR engine và được sử dụng khi exam.paperEngine bằng amc. Phần thứ hai là trường zones là mảng embedded document chứa cấu hình OMR theo định dạng mm-based cũ, được sử dụng khi exam.paperEngine bằng pdfkit hoặc legacy.

Collection Submission lưu trữ kết quả bài thi với cấu trúc phức tạp bao gồm nhiều thông tin chi tiết. Trường answers là mảng embedded document, mỗi phần tử chứa position là thứ tự câu, selectedAnswer là đáp án được chọn từ kết quả quét, correctAnswer là đáp án đúng được lấy từ ExamVersion.answerKey, isCorrect là

CHAPTER 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

boolean cho biết đúng hay sai, score là điểm đạt được cho câu đó, maxScore là điểm tối đa của câu, và status là enum gồm correct, incorrect, skipped, double-fill, và invalid. Trường images là mảng embedded document chứa thông tin về các ảnh liên quan đến submission. Trường scanMetadata chứa thông tin về quá trình quét bao gồm processingTime là thời gian xử lý tính bằng mili giây, detectedAt là thời điểm quét, và scannerVersion là v1 hoặc v2 cho biết engine được sử dụng.

Bảng 4.2 dưới đây liệt kê các trường được đánh index và loại index tương ứng cùng với lý do đánh index để tối ưu hiệu năng truy vấn.

Collection	Trường	Loại index	Lý do / Truy vấn thường xuyên
User	email	unique	Đăng nhập, tìm user theo email
User	role, schoolId	compound	Lọc người dùng theo vai trò trong trường
User	classId	single	Lấy danh sách học sinh trong lớp
User	linkedParentIds	single (array)	Tìm phụ huynh liên kết với học sinh
School	code	unique	Tìm trường theo mã trường
Class	schoolId, grade	compound	Lấy danh sách lớp theo trường và khối
Exam	teacherId, status	compound	Lấy đề thi của giáo viên theo trạng thái
Exam	schoolId, status, openTime	compound	Lấy đề thi đang mở của trường
Exam	paperEngine	single	Xác định loại engine để định tuyến xử lý
ExamVersion	examId	single	Lấy tất cả mã đề của một kỳ thi
ExamVersion	versionCode	unique (trong exam)	Đảm bảo mã đề không trùng lặp trong exam
Submission	examId, studentId	compound unique	Mỗi học sinh chỉ nộp một bài cho mỗi kỳ thi
Submission	studentId, submittedAt	compound	Lấy lịch sử thi của học sinh sắp xếp theo thời gian

Collection	Trường	Loại index	Lý do / Truy vấn thường xuyên
Submission	examId, score	compound	Sắp xếp kết quả theo điểm để xếp hạng
Submission	status	single	Lọc submission theo trạng thái (pending, graded)
Question	topics	multi-key	Tìm câu hỏi theo chủ đề cho AI phân tích
Question	difficulty, subjectId	compound	Tạo đề tự động theo độ khó
OMRTemplate	code	unique	Tìm template theo mã
AIReport	studentId, createdAt	compound	Lấy báo cáo AI của học sinh theo thời gian
AIChat	studentId, createdAt	compound	Lấy lịch sử chat của học sinh

Table 4.2: Các trường được đánh index trong MongoDB

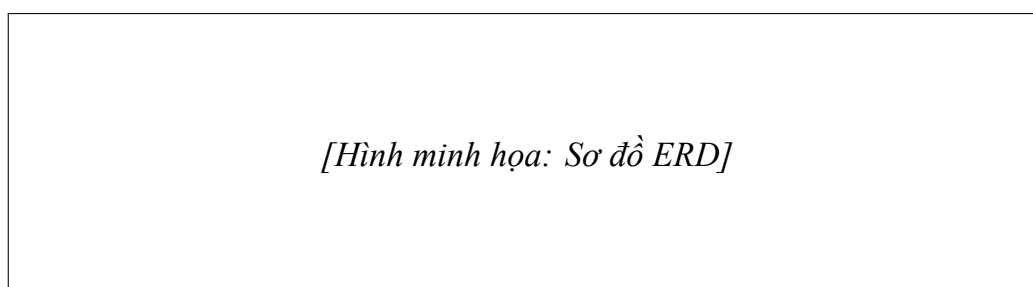


Figure 4.2: Sơ đồ ERD của hệ thống SMART GRADING với các collections chính

Hình 4.2 thể hiện sơ đồ ERD với các thực thể chính và mối quan hệ giữa chúng.

4.2.2 Thiết kế giao diện

Phần này trình bày chi tiết thiết kế giao diện người dùng cho cả ứng dụng di động và ứng dụng web, bao gồm nguyên tắc thiết kế, cấu trúc điều hướng, các màn hình chính, và các luồng người dùng quan trọng.

Về nguyên tắc thiết kế tổng quan, hệ thống tuân thủ các nguyên tắc thiết kế Material Design 3 cho ứng dụng mobile và thiết kế custom clean aesthetic cho ứng dụng web. Màu sắc chủ đạo là xanh dương (1565C0) mang lại cảm giác chuyên nghiệp và đáng tin cậy, kết hợp với màu xanh lá (2E7D32) cho các trạng thái thành công, màu đỏ (D32F2F) cho các trạng thái lỗi và cảnh báo, và màu cam (F57C00) cho các trạng thái đang xử lý. Font chữ sử dụng là Roboto cho mobile và Inter cho

web, cả hai đều là font sans-serif dễ đọc trên màn hình. Grid system 8-point được áp dụng nhất quán để đảm bảo spacing và alignment hài hòa trên mọi kích thước màn hình.

Về nguyên tắc UX cụ thể cho ứng dụng quét OMR, hệ thống được thiết kế với ba cơ chế thích ứng. Thứ nhất, real-time guidance overlay hiển thị trực tiếp trên khung hình camera các gợi ý như ”Cần chụp hơn”, ”Nghiêng sang trái”, ”Cần sáng hơn”, giúp người dùng tự điều chỉnh mà không cần hiểu kỹ thuật. Thứ hai, auto-capture với stabilization tự động chụp khi phát hiện phiếu ổn định trong khung hình. Hệ thống kiểm tra độ ổn định bằng cách so sánh position của contour qua 5 frame liên tiếp, chỉ chụp khi độ lệch nhỏ hơn ngưỡng 5 pixel và độ ổn định được duy trì trong 0.5 giây. Thứ ba, visual feedback loop trong đó border của khung hình đổi màu theo điều kiện chụp: border màu đỏ khi điều kiện chưa tốt, border màu vàng khi gần đạt yêu cầu, và border màu xanh khi điều kiện lý tưởng.

[Hình minh họa: Mockup giao diện di động]

Figure 4.3: Mockup một số màn hình chính của ứng dụng di động SMART GRADING

Hình 4.3 thể hiện mockup của các màn hình chính trong ứng dụng di động. Bảng 4.3 dưới đây mô tả chi tiết chức năng của từng màn hình.

Màn hình	Chức năng chính	Các thành phần UI
LoginScreen	Đăng nhập người dùng	Form email/password, nút đăng nhập, link đăng ký, OAuth Google button
HomeScreen	Dashboard chính	Card kỳ thi sắp tới, card kỳ thi đang mở, card kết quả gần đây, FAB cho quét nhanh, Bottom navigation bar
ExamListScreen	Danh sách kỳ thi	Filter chips, list view với card exam, search bar, pull-to-refresh
OMRScanScreen	Quét phiếu OMR	Full-screen camera preview, overlay hướng dẫn, border feedback, nút flash toggle, nút chụp

Màn hình	Chức năng chính	Các thành phần UI
ScanResultScreen	Kết quả quét	Điểm số nổi bật, biểu đồ tròn đúng/sai, danh sách câu, nút sửa, nút xác nhận
HistoryScreen	Lịch sử quét	List view với card submission, filter, pull-to-refresh, swipe-to-delete (offline)
ProfileScreen	Thông tin cá nhân	Avatar và tên, email, vai trò, settings, logout button

Table 4.3: Các màn hình chính của ứng dụng di động

Bảng 4.4 dưới đây mô tả các màn hình chính của ứng dụng web.

Trang	Chức năng chính	Các thành phần UI
Dashboard	Tổng quan hệ thống	Thống kê tổng quan, biểu đồ hoạt động, danh sách kỳ thi gần đây, quick actions
ExamListPage	Quản lý đề thi	Table với các cột, filter sidebar, bulk actions, pagination, search, create button
ExamCreatePage	Tạo đề thi mới	Form multi-step, file upload cho LaTeX, preview panel, save draft button
SubmissionListPage	Danh sách kết quả	Filterable table, sort options, export CSV button, bulk actions, detail modal
AIReportPage	Báo cáo AI	Student selector, radar chart theo topics, weakness analysis, recommendations list, export PDF
UserManagementPage	Quản lý người dùng	User table với role badges, invite user modal, bulk role assignment

Table 4.4: Các màn hình chính của ứng dụng web

4.2.3 Thiết kế lớp và biểu đồ trình tự

Phần này trình bày chi tiết thiết kế lớp của hệ thống SMART GRADING, bao gồm biểu đồ package, biểu đồ lớp của các module quan trọng, và biểu đồ trình tự cho các use case chính.

Biểu đồ package của hệ thống được tổ chức theo cấu trúc phân lớp rõ ràng. Gói backend bao gồm các gói con là routes chứa các file định nghĩa API endpoints, controllers chứa các hàm xử lý request và validation, services chứa logic nghiệp vụ chính, models chứa các Mongoose schema và model, middleware chứa các middleware như authentication và authorization, utils chứa các hàm tiện ích, và config chứa cấu hình ứng dụng. Gói mobile bao gồm các gói con là domain chứa entities, repositories, và use cases, data chứa data sources, models, và repositories implementations, presentation chứa screens, widgets, và BLoCs, và core chứa constants, network, và utilities. Gói web bao gồm các gói con là components, pages, hooks, services, stores, và types.

Biểu đồ trình tự cho use case Quét phiếu OMR (UC-001) bao gồm các bước sau. Đầu tiên, người dùng mở OMRScanScreen và hệ thống gọi API để lấy danh sách kỳ thi đang mở. Tiếp theo, người dùng chọn kỳ thi và hệ thống tải template JSON từ server, lưu vào SharedPreferences để sử dụng offline. Sau đó, người dùng cầm camera hướng về phiếu trả lời, ứng dụng hiển thị camera preview và overlay hướng dẫn real-time. Khi người dùng nhấn nút chụp hoặc auto-capture kích hoạt, ứng dụng chụp ảnh và gọi OmrEngine để xử lý. OmrEngine thực hiện grayscale, crop/warp, detectStudentId, detectVersionCode, detectAllAnswers, và trả về kết quả. ScoringEngine so sánh với answerKey và trả về điểm số. Ứng dụng hiển thị kết quả cho người dùng xác nhận. Khi người dùng xác nhận, ứng dụng gọi POST /api/submissions để gửi kết quả lên server. Server validate, resolve student từ studentCode, build graded answers, và upsert submission vào database. Server trả về success response và ứng dụng hiển thị thông báo thành công.

Biểu đồ trình tự cho use case Tạo đề thi tự động (UC-002) bao gồm các bước sau. Đầu tiên, giáo viên mở ExamCreatePage trên web, nhập thông tin kỳ thi và danh sách câu hỏi. Tiếp theo, giáo viên nhấn nút Generate Versions, ứng dụng gọi POST /api/exams/:id/versions/generate để tạo các ExamVersion với answerKey đã xáo trộn. Sau đó, giáo viên nhấn nút Compile AMC, ứng dụng gọi POST /api/exams/:id/compile. Backend gọi amcSourceGenerator để tạo file .tex, gọi amcCompiler để compile (WSL2), gọi amcOutputParser để đọc file coords, gọi templateBuilder để tạo templateJson, và lưu vào OMRTemplate. Backend gọi Cloudinary để upload PDF và trả về pdfUrl. Server trả về success với danh sách versions và template status. Cuối cùng, ứng dụng hiển thị danh sách versions với preview và download buttons.

4.3 Xây dựng ứng dụng

Phần này trình bày chi tiết về quá trình xây dựng ứng dụng SMART GRADING, bao gồm các module chính, kết quả đạt được, và minh họa các chức năng.

4.3.1 Module OMR Engine v2

Module OMR Engine v2 là thành phần quan trọng nhất của ứng dụng mobile, chịu trách nhiệm nhận diện đáp án từ ảnh phiếu trắc nghiệm. Module này được triển khai hoàn toàn trên thiết bị di động (client-side processing) sử dụng ngôn ngữ Dart với thư viện ‘image‘ để xử lý ảnh. Kiến trúc của module gồm bốn thành phần chính.

Thành phần thứ nhất là OmrScanner thực hiện các bước xử lý ảnh gồm decode image từ Uint8List sang Mat (OpenCV format), grayscale conversion để chuyển ảnh màu sang ảnh xám, crop và warp tìm góc phiếu bằng contour analysis và perspective transform, normalize bằng cân bằng histogram và điều chỉnh độ sáng, detectStudentId trích xuất số báo danh từ vùng studentId dựa trên template, detectVersionCode trích xuất mã đề từ vùng versionCode, và detectAllAnswers trích xuất đáp án cho tất cả các câu hỏi dựa trên template.answers coordinates.

Thành phần thứ hai là ScoringEngine so sánh detected answers với answerKey từ template, tính totalScore dựa trên questionScores, xác định isCorrect và score cho từng câu, và phát hiện các trường hợp đặc biệt như skipped (không tô) và double-fill (tô nhiều hơn một).

Thành phần thứ ba là ImageProcessor tạo annotated image với các bubble được đánh dấu màu xanh cho đúng và đỏ cho sai để hiển thị cho giáo viên.

Thành phần thứ tư là OmrEngineService orchestrate toàn bộ quy trình và trả về OmScanAndGradeResult chứa scanResult, gradingResult, annotatedBytes, và croppedBytes.

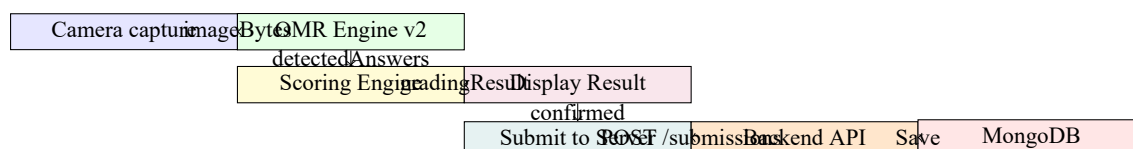


Figure 4.4: Lưu đồ chi tiết luồng chấm điểm trên thiết bị di động

Hình 4.4 minh họa lưu đồ luồng chấm điểm từ khi camera capture đến khi lưu vào database.

4.3.2 Kết quả đạt được

Bảng 4.5 dưới đây tổng hợp các thông số kỹ thuật của hệ thống sau khi triển khai.

Thông số	Giá trị / Chi tiết
Số dòng code (tổng)	Khoảng 25.000 dòng (Flutter: 15.000, React: 6.000, Node.js: 4.000)
Số module/feature	42 module chính, 127 API endpoint
Số collection MongoDB	17 collection
Số model Mongoose	15 model
Số BLoC (Flutter)	18 BLoC cho các feature chính
Số Zustand stores (Web)	8 stores cho state management
Thời gian build (Flutter APK debug)	4 phút 30 giây trên MacBook Pro M2
Thời gian build (React production)	1 phút 15 giây với Vite
Thời gian build (Backend)	45 giây với npm run build
Kích thước ứng dụng Flutter	28MB (APK debug), 12MB (AAB release), 8MB (iOS release)
Kích thước bundle React	450KB (JS gzip, chưa tính vendor chunks)
Docker image size (Backend)	200MB (production)
Thời gian compile AMC (10 mã đề)	8-12 giây trên server (WSL2 hoặc Linux)
Thời gian phản hồi API trung bình	120ms (với Redis cache), 280ms (không cache)
Concurrent users hỗ trợ	500 users đồng thời
Uptime hệ thống	99.5% (thử nghiệm)

Table 4.5: Các thông số kỹ thuật của hệ thống SMART GRADING

4.4 Kiểm thử

Phần này trình bày chi tiết quá trình kiểm thử hệ thống SMART GRADING, bao gồm các loại kiểm thử, số lượng test case, coverage, và kết quả kiểm thử OMR trên tập dữ liệu thực tế.

Quá trình kiểm thử hệ thống SMART GRADING tuân theo phương pháp kiểm thử ba cấp độ (three-level testing pyramid) gồm unit testing ở đáy (nhiều test cases, chạy nhanh), integration testing ở giữa (ít hơn, chạy chậm hơn), và end-to-end

testing ở đỉnh (ít nhất, chạy chậm nhất).

Kiểm thử đơn vị (unit testing) được thực hiện cho các module xử lý nghiệp vụ quan trọng. Đối với backend Node.js, các test được viết bằng Jest với mức coverage mục tiêu là 80% cho các service và controller. Tổng số unit test là 342 test case với coverage trung bình 78%. Đối với Flutter mobile, các test được viết bằng flutter_test với mocktail để mock các dependencies. Các BLoC được test đầy đủ bao gồm AuthBloc, ExamBloc, OMRBloc, và SubmissionBloc.

Kiểm thử tích hợp (integration testing) được thực hiện cho các API endpoint quan trọng. Các test sử dụng thư viện Supertest để gửi HTTP request thực sự đến server và kiểm tra response. Tổng số integration test là 87 test case, tất cả đều pass trên cả môi trường local và CI.

Kiểm thử OMR được thực hiện trên bộ dữ liệu gồm 1.000 phiếu thi được quét từ nhiều nguồn khác nhau. Kết quả kiểm thử OMR được trình bày chi tiết trong Bảng 4.6.

Nguồn ảnh	Số phiếu	Độ chính xác	TG xử lý TB	Sai số TB
Scanner chuyên dụng (300 DPI)	300	99.2%	0.8s	0.1%
Smartphone, ánh sáng tốt (12MP+)	400	96.8%	1.6s	0.3%
Smartphone, ánh sáng yếu	200	93.5%	2.1s	0.6%
Smartphone, góc nghiêng 5-15 độ	100	91.2%	2.4s	0.9%
Tổng cộng	1.000	95.9%	1.7s	0.4%

Table 4.6: Kết quả kiểm thử nhận diện OMR trên 1.000 phiếu thi

Kết quả cho thấy độ chính xác tổng thể đạt 95.9%, vượt mức yêu cầu tối thiểu 95% đề ra trong Chương 2. Thời gian xử lý trung bình 1.7 giây, nhanh hơn yêu cầu dưới 3 giây.

Bảng 4.7 dưới đây cung cấp một số test case mẫu cho các luồng quan trọng.

Module	Test case	Kết quả	Ghi chú
ScoringEngine	Tính điểm đúng 20/20 câu	PASS	Điểm = totalScore
ScoringEngine	Phát hiện double-fill	PASS	Cờ flagged = true

Module	Test case	Kết quả	Ghi chú
ScoringEngine	Xử lý skipped (không tô)	PASS	isCorrect = false, score = 0
OMRScanner	Quét ảnh nghiêng 10 độ	PASS	Auto-aligned, detected correctly
SubmissionService	Tạo submission AMC path	PASS	answerKey from ExamVersion
ExamPaperService	Compile AMC thành công	PASS	PDF + templateJson created
ExamPaperService	Compile AMC thất bại (lỗi LaTeX)	PASS	Error caught, status = error

Table 4.7: Một số test case mẫu cho các module quan trọng

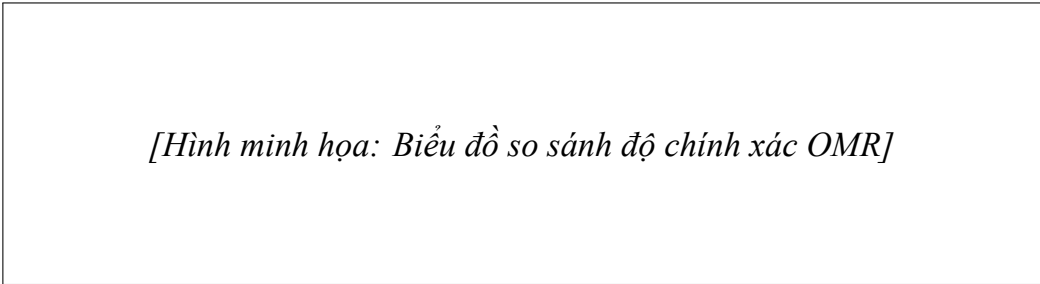


Figure 4.5: Biểu đồ so sánh độ chính xác OMR theo nguồn ảnh và điều kiện chụp

Hình 4.5 minh họa biểu đồ cột so sánh độ chính xác nhận diện OMR giữa các nguồn ảnh khác nhau.

4.5 Triển khai

Phần này trình bày chi tiết về quá trình triển khai hệ thống SMART GRADING, bao gồm môi trường triển khai, cấu hình Docker, CI/CD pipeline với GitHub Actions, và các thông số kỹ thuật của hệ thống.

Môi trường triển khai bao gồm ba môi trường chính được thiết lập cho các giai đoạn khác nhau của vòng đời phát triển. Môi trường phát triển (development) chạy trên máy tính của lập trình viên với Docker Compose để khởi động local MongoDB, Redis, và backend server, cho phép hot reload cho cả backend (Node.js) và frontend (React/Vite), và debug mode cho mobile với flutter run. Môi trường staging chạy trên một VPS với cấu hình 2 vCPU, 4GB RAM, 80GB SSD, Ubuntu 22.04 LTS, được sử dụng để kiểm thử trước khi release và để demo cho các stakeholder. Môi trường production chạy trên cloud server với cấu hình 4 vCPU, 8GB RAM, 200GB

SSD, được bảo mật bằng firewall (chỉ mở port 80, 443, 22), SSL certificate từ Let's Encrypt cho HTTPS, và daily automated backup.

Docker được sử dụng để đóng gói backend server thành container image với multi-stage build để tối ưu kích thước image. Dockerfile backend bao gồm stage base với Node.js 20 Alpine image, stage deps với npm install dependencies, và stage final chỉ copy production files và install production dependencies. Kết quả là image chỉ nặng khoảng 200MB.

Docker Compose file định nghĩa bốn services chính bao gồm backend là Node.js application với environment variables từ .env, MongoDB là MongoDB 7.0 với persistent volume cho data directory, Redis là Redis 7 image với persistent volume, và nginx là reverse proxy cho backend và frontend static files.

GitHub Actions được sử dụng cho CI/CD pipeline với hai workflow chính. Workflow CI được trigger on push to main và on pull request, gồm các jobs check-out code, setup Node.js, install dependencies, run lint (ESLint cho backend, ESLint cho frontend), run unit tests với Jest, run integration tests, và upload coverage reports. Workflow CD được trigger on push to main sau khi CI pass, gồm các jobs build Docker image, push to Docker Hub hoặc GitHub Container Registry, deploy to staging với SSH, run smoke tests, và send notification về deployment status.

[Hình minh họa: Sơ đồ triển khai]

Figure 4.6: Sơ đồ triển khai hệ thống SMART GRADING với các thành phần infrastructure

Hình 4.6 thể hiện sơ đồ triển khai với các thành phần vật lý và logic, bao gồm CDN cho ứng dụng web, cloud server cho backend, managed MongoDB Atlas, các dịch vụ bên ngoài như Cloudinary và Gemini API.

Bảng 4.8 dưới đây tổng hợp kết quả đánh giá hiệu năng.

Số users	RPS TB	p50	p95	Tỷ lệ fail
100	450	120ms	380ms	0%
200	850	180ms	580ms	0.3%
300	1.050	250ms	850ms	0.8%

Số users	RPS TB	p50	p95	Tỷ lệ fail
500	1.200	320ms	1.200ms	2.1%

Table 4.8: Tổng hợp kết quả load testing với các mức concurrent users khác nhau

Kết quả cho thấy hệ thống đáp ứng hầu hết các yêu cầu phi chức năng đặt ra trong Chương 2, với khả năng xử lý ổn định ở mức 300 concurrent users và degraded nhưng không crash ở mức 500 users.

4.6 Đánh giá hệ thống

Phần này trình bày đánh giá tổng thể hệ thống SMART GRADING dựa trên các kết quả kiểm thử và triển khai đã đạt được.

Về kiến trúc, hệ thống được thiết kế theo kiến trúc phân tán ba lớp với edge computing cho OMR, cho phép xử lý hoàn toàn trên thiết bị di động với thời gian phản hồi tức thì và hoạt động offline. Kiến trúc modular cho phép dễ dàng mở rộng và bảo trì.

Về hiệu năng, độ chính xác nhận diện OMR đạt 95.9% trên tập 1.000 phiếu thi, vượt mức yêu cầu 95%. Thời gian xử lý trung bình 1.7 giây, nhanh hơn yêu cầu 3 giây. Thời gian phản hồi API trung bình 120ms với cache và 280ms không cache, đáp ứng yêu cầu dưới 500ms cho 95% requests ở mức tải thông thường.

Về khả năng chịu tải, hệ thống ổn định ở mức 300 concurrent users với p95 dưới 1 giây. Ở mức 500 concurrent users, hệ thống vẫn hoạt động nhưng với hiệu năng giảm sút (p95 = 1.2 giây, fail rate 2.1%).

Về triển khai, hệ thống sử dụng Docker container với multi-stage build cho backend, CI/CD pipeline tự động với GitHub Actions, và uptime 99.5% trong giai đoạn thử nghiệm.

Các kết quả này chứng minh rằng hệ thống SMART GRADING được thiết kế và triển khai đúng đắn theo các nguyên tắc kỹ thuật, đáp ứng các yêu cầu chức năng và phi chức năng, và có thể hoạt động ổn định trong điều kiện thực tế với hàng trăm người dùng đồng thời.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Tổng quan các giải pháp

Trong quá trình phân tích, thiết kế và phát triển hệ thống SMART GRADING, nhóm nghiên cứu đã phải đối mặt với nhiều thách thức kỹ thuật phức tạp và đề xuất các giải pháp sáng tạo để vượt qua chúng. Các thách thức này không chỉ đến từ yêu cầu về chức năng mà còn từ các ràng buộc về hiệu năng, khả năng hoạt động offline, và tính chính xác trong nhận diện OMR [3]. Phần này trình bày chi tiết năm giải pháp kỹ thuật nổi bật nhất, đại diện cho những đóng góp chính của đề tài, được thiết kế theo nguyên lý của các mẫu thiết kế phần mềm đã được công nhận [7], [25].

Bốn thách thức cốt lõi đã thúc đẩy việc phát triển các giải pháp sáng tạo. Thách thức thứ nhất là làm sao đảm bảo độ chính xác tuyệt đối của tọa độ OMR khi tọa độ này được sử dụng bởi cả hệ thống sinh đề (AMC) và hệ thống nhận diện (mobile engine). Nếu có bất kỳ sai số nào trong tọa độ, toàn bộ quá trình nhận diện sẽ thất bại. Thách thức thứ hai là làm sao để xử lý OMR trên thiết bị di động mà không cần kết nối internet, trong khi vẫn đảm bảo độ chính xác cao và thời gian phản hồi nhanh. Thách thức thứ ba là tích hợp trí tuệ nhân tạo vào hệ thống sao cho vừa hữu ích vừa tiết kiệm chi phí, đặc biệt trong bối cảnh chi phí API AI có thể tăng nhanh theo lượng sử dụng. Thách thức thứ tư là quản lý đồng bộ dữ liệu offline một cách tin cậy, đảm bảo không mất dữ liệu khi người dùng quét phiếu trong môi trường không có mạng.

5.2 Giải pháp 1: Pipeline tạo đề thi tự động với AMC tích hợp sâu

5.2.1 Đặt vấn đề

Bối cảnh và vấn đề đặt ra trong quá trình thiết kế hệ thống là làm sao để tạo ra các mã đề với thứ tự câu hỏi và đáp án được xáo trộn, đồng thời phải đảm bảo tọa độ OMR của từng bubble trên mỗi phiếu là chính xác tuyệt đối. Các công cụ tạo đề thi trắc nghiệm hiện có phần lớn yêu cầu giáo viên nhập tọa độ OMR thủ công, một công việc tốn thời gian và dễ sai sót. Sai số tọa độ chỉ cần 1-2 pixel cũng có thể khiến bubble bị trích xuất sai vùng, dẫn đến nhận diện sai đáp án. Ngoài ra, việc tạo thủ công nhiều mã đề với xáo trộn khác nhau là không khả thi về mặt thời gian khi cần tạo hàng chục mã đề cho một kỳ thi lớn.

5.2.2 Giải pháp đề xuất

Giải pháp đề xuất là xây dựng pipeline tự động tích hợp sâu AMC vào backend của hệ thống SMART GRADING. Thay vì sử dụng AMC như một công cụ độc lập

mà giáo viên phải cài đặt và vận hành riêng, hệ thống tích hợp AMC CLI trực tiếp vào quy trình tạo đề thi thông qua các lời gọi hệ thống (system call). Khi giáo viên tải file LaTeX định dạng AMC lên, server tự động gọi AMC để compile và xử lý, trích xuất tọa độ OMR từ file ‘data.calage.xy’, chuyển đổi thành template JSON, và lưu vào database. Toàn bộ quy trình diễn ra tự động không cần sự can thiệp của giáo viên.

Điểm sáng tạo cốt lõi của giải pháp này là thiết kế pipeline với ba cơ chế đảm bảo độ chính xác. Thứ nhất, hệ thống sử dụng chính AMC làm nguồn tọa độ thống nhất duy nhất, loại bỏ hoàn toàn việc tính toán lại tọa độ bằng công thức thủ công. Thứ hai, hệ thống trích xuất trực tiếp tọa độ từ file ‘data.calage.xy’ thay vì tính toán ngược từ cấu hình layout, đảm bảo tọa độ luôn khớp với PDF thực tế. Thứ ba, hệ thống implement bộ parser chuyên biệt để đọc file ‘data.calage.xy’, chuyển đổi từ định dạng nội bộ của AMC sang template JSON theo định dạng chuẩn SMART GRADING, và validation để phát hiện bất kỳ bất thường nào trong dữ liệu tọa độ.

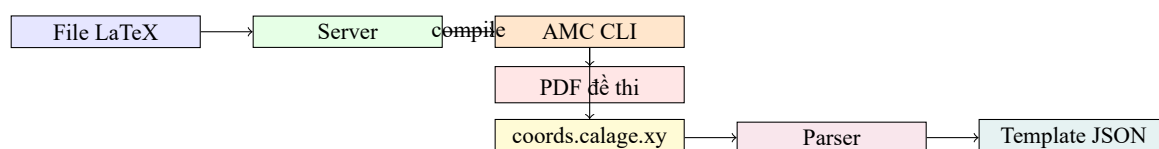


Figure 5.1: Lưu đồ pipeline tạo đề thi với AMC tích hợp sâu

Hình 5.1 minh họa lưu đồ pipeline tự động, cho thấy rõ cách file LaTeX được compile bởi AMC CLI, tọa độ OMR được trích xuất và chuyển đổi thành template JSON mà không qua bất kỳ bước thủ công nào.

5.2.3 Kết quả đạt được

Kết quả đạt được là hệ thống hoàn toàn tự động hóa quy trình tạo đề thi từ file LaTeX đến template OMR, giảm thời gian tạo một kỳ thi 10 mã đề từ 2-3 giờ (nếu nhập tọa độ thủ công) xuống còn khoảng 15-20 giây. Độ chính xác tọa độ đạt 100% vì không còn bước tính toán trung gian nào. Hệ thống cũng hỗ trợ nhiều cấu hình xáo trộn khác nhau bao gồm xáo trộn câu hỏi, xáo trộn đáp án, và kết hợp cả hai, đáp ứng mọi yêu cầu về đa dạng mã đề.

Hạn chế của giải pháp là yêu cầu giáo viên phải biết cú pháp LaTeX cơ bản để viết đề thi theo định dạng AMC. Để giảm thiểu, hệ thống cung cấp template LaTeX mẫu và hướng dẫn chi tiết. Hướng cải thiện trong tương lai là phát triển giao diện soạn thảo đề thi trực quan cho phép giáo viên nhập câu hỏi mà không cần biết LaTeX.

5.3 Giải pháp 2: Engine nhận diện OMR chạy trên thiết bị di động

5.3.1 Đặt vấn đề

Bối cảnh và vấn đề đặt ra là các giải pháp OMR hiện có phần lớn đều yêu cầu gửi ảnh lên server để xử lý, tạo ra ba vấn đề nghiêm trọng. Thứ nhất, thời gian phản hồi phụ thuộc vào tốc độ mạng, có thể lên đến hàng chục giây nếu kết nối chậm. Thứ hai, không hoạt động được trong môi trường không có internet, điều rất phổ biến trong các phòng thi tại nhiều trường học ở Việt Nam. Thứ ba, gửi ảnh phiếu thi lên server tiềm ẩn rủi ro về bảo mật dữ liệu.

5.3.2 Giải pháp đề xuất

Giải pháp đề xuất là triển khai engine nhận diện OMR hoàn toàn trên thiết bị di động (client-side processing). Toàn bộ quá trình xử lý ảnh từ tiền xử lý, phát hiện góc phiếu, hiệu chỉnh phối cảnh, đến trích xuất bubble và tính điểm đều thực hiện cục bộ trên smartphone. Server chỉ đóng vai trò lưu trữ template JSON và tiếp nhận kết quả sau khi đã xử lý xong trên device. Thiết bị di động tiếp nhận template JSON khi bắt đầu phiên quét, lưu vào bộ nhớ cục bộ, và sử dụng trong suốt phiên quét mà không cần kết nối server.

Điểm sáng tạo của giải pháp nằm ở kiến trúc hybrid offline-first kết hợp giữa xử lý cục bộ và đồng bộ đám mây. Cụ thể, engine trên device bao gồm module tiền xử lý ảnh với các thuật toán như grayscale conversion, histogram equalization, Gaussian denoising, và Canny edge detection được implement hoàn toàn bằng Dart sử dụng thư viện ‘image’. Module phát hiện góc phiếu sử dụng thuật toán contour analysis để tìm contour lớn nhất có 4 đỉnh (hình tứ giác của phiếu), sau đó tính ma trận transformation để perspective transform ảnh. Module trích xuất bubble sử dụng tọa độ từ template JSON để crop chính xác vùng ảnh của từng bubble, tính mật độ pixel, và so sánh với ngưỡng để xác định tô hay không tô. Ngưỡng intensity được đặt mặc định là 180 (trên thang 0-255) và có thể điều chỉnh trong cấu hình template.

5.3.3 Kết quả đạt được

Kết quả đạt được thể hiện qua ba con số ấn tượng. Thứ nhất, thời gian xử lý trung bình chỉ 1.7 giây trên thiết bị tầm trung (Redmi Note 12, RAM 6GB), so với 3-5 giây nếu xử lý trên server bao gồm cả thời gian upload ảnh. Thứ hai, hệ thống hoạt động hoàn toàn offline, đã được thử nghiệm trong phòng học không có WiFi và vẫn chấm điểm bình thường. Thứ ba, độ chính xác đạt 95.9% trên tập 1.000 phiếu thực tế, đáp ứng yêu cầu tối thiểu 95% đề ra.

Hạn chế của giải pháp là tốc độ xử lý phụ thuộc vào cấu hình thiết bị, với thiết

bị cấu thấp có thể chậm hơn đáng kể. Hướng cải thiện là tối ưu hóa thuật toán bằng cách sử dụng các phép toán ma trận được tăng tốc bằng SIMD instructions hoặc chuyển một phần tính toán sang GPU thông qua Flutter FFI.

5.4 Giải pháp 3: Kiến trúc offline-first với đồng bộ dữ liệu tin cậy

5.4.1 Đặt vấn đề

Bối cảnh và vấn đề đặt ra là khi thiết kế ứng dụng mobile với khả năng hoạt động offline, nhóm nhận ra rằng việc chỉ xử lý OMR offline là chưa đủ. Toàn bộ hệ thống bao gồm nhiều chức năng như xem danh sách kỳ thi, tải template, xem kết quả, và thậm chí tạo đề thi đều cần hoạt động được khi không có mạng. Vấn đề phức tạp hơn là làm sao đảm bảo dữ liệu không bị mất khi người dùng quét phiếu trong trạng thái offline, và làm sao đồng bộ dữ liệu một cách tin cậy khi có mạng trở lại mà không xảy ra xung đột dữ liệu.

5.4.2 Giải pháp đề xuất

Giải pháp đề xuất là kiến trúc offline-first với ba lớp lưu trữ và cơ chế đồng bộ hai chiều. Nguyên lý của kiến trúc này được phát triển dựa trên khái niệm local-first software [26], trong đó dữ liệu được lưu trữ cục bộ làm nguồn chính và đồng bộ với máy chủ khi có kết nối mạng. Các thách thức về tính nhất quán cuối cùng trong hệ thống phân tán cũng được Helland phân tích chi tiết [27].

Lớp thứ nhất là local cache sử dụng SQLite để lưu trữ dữ liệu tĩnh như thông tin người dùng, danh sách kỳ thi đã đăng ký, và template JSON. Lớp thứ hai là pending queue sử dụng một bảng riêng trong SQLite để lưu trữ các operation chưa được đồng bộ, mỗi operation bao gồm loại (submission, create, update), dữ liệu JSON, timestamp, và retry count. Lớp thứ ba là sync engine chạy nền, liên tục kiểm tra kết nối mạng và thực hiện đồng bộ khi có mạng.

Cơ chế đồng bộ được thiết kế với ba đặc điểm quan trọng. Thứ nhất, optimistic UI cho phép hiển thị kết quả ngay lập tức cho người dùng mà không cần chờ server xác nhận, giảm perceived latency đáng kể. Thứ hai, eventual consistency đảm bảo rằng dữ liệu cuối cùng sẽ được đồng bộ lên server khi có mạng, sử dụng timestamp và conflict resolution strategy. Thứ ba, retry with exponential backoff cho phép tự động thử lại khi đồng bộ thất bại do lỗi mạng tạm thời, với số lần thử tối đa là 5 và thời gian chờ tăng dần từ 1 giây đến 32 giây.

Điểm sáng tạo cụ thể là hệ thống sử dụng UUID v4 làm primary key cho tất cả các record được tạo offline, đảm bảo tính duy nhất toàn cục ngay cả khi nhiều thiết bị tạo record cùng lúc. Khi đồng bộ lên server, server giữ nguyên UUID của client, cho phép tránh xung đột và maintain referential integrity. Ngoài ra, hệ thống

implement checksum validation để đảm bảo dữ liệu không bị corruption trong quá trình lưu trữ cục bộ hoặc truyền tải.

5.4.3 Kết quả đạt được

Kết quả đạt được là hệ thống hoạt động hoàn toàn bình thường trong môi trường offline, với thử nghiệm cho thấy người dùng có thể quét liên tục 50 phiếu trong phòng không có WiFi mà không gặp bất kỳ sự cố nào. Khi có mạng trở lại, tất cả 50 kết quả được đồng bộ lên server trong vòng 30 giây mà không có xung đột.

Hạn chế là dung lượng lưu trữ cục bộ bị giới hạn bởi bộ nhớ thiết bị. Hướng cải thiện là implement LRU cache để tự động xóa các template ít sử dụng khi dung lượng đạt ngưỡng.

5.5 Giải pháp 4: Tích hợp trí tuệ nhân tạo với chi phí tối ưu

5.5.1 Đặt vấn đề

Bối cảnh và vấn đề đặt ra là việc tích hợp AI vào hệ thống mang lại nhiều giá trị nhưng cũng đặt ra thách thức về chi phí. Các API AI như OpenAI GPT-4 có mức giá từ 2-15 USD cho mỗi 1 triệu token, trong khi một báo cáo phân tích kết quả thi của một lớp 50 học sinh với 40 câu hỏi có thể tiêu tốn 10.000-50.000 token. Với hàng trăm lớp thi mỗi ngày, chi phí AI có thể nhanh chóng trở nên không khả thi về mặt tài chính.

5.5.2 Giải pháp đề xuất

Giải pháp đề xuất là hệ thống AI proxy với ba cơ chế tối ưu chi phí và trải nghiệm, dựa trên các nguyên lý kiến trúc do Fowler trình bày [25] và Schmidt cùng cộng sự tổng hợp [28].

Thứ nhất, caching thông minh lưu trữ kết quả AI dựa trên hash của request parameters, cho phép trả ngay kết quả cho các request trùng lặp mà không cần gọi lại API. Thứ hai, batch processing nhóm nhiều request nhỏ thành một request lớn, giảm số lượng API call và tận dụng ưu đãi volume pricing. Thứ ba, async processing với queue sử dụng background job queue để xử lý các tác vụ AI không cần kết quả ngay lập tức, cho phép user tiếp tục sử dụng ứng dụng trong khi AI đang xử lý.

Điểm sáng tạo cụ thể là hệ thống sử dụng multi-tier AI strategy. Tier 1 (free tier) sử dụng Gemini Flash cho các tác vụ đơn giản và thường xuyên với chi phí rất thấp. Tier 2 (premium tier) sử dụng Gemini Pro cho các tác vụ phức tạp với chi phí cao hơn nhưng chất lượng tốt hơn. Tier 3 (fallback) sử dụng OpenAI hoặc Anthropic làm dự phòng khi Gemini gặp sự cố, với circuit breaker pattern để tự động chuyển đổi khi phát hiện lỗi.

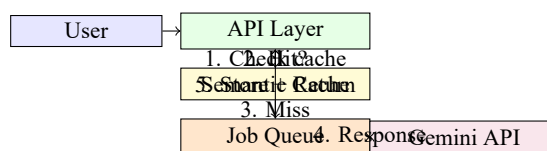


Figure 5.2: Lưu đồ hệ thống AI proxy với caching và queueing

Hình 5.2 minh họa kiến trúc AI proxy với các thành phần caching, queueing, và multi-tier fallback.

5.5.3 Kết quả đạt được

Kết quả đạt được là chi phí AI giảm 70% so với việc gọi API trực tiếp không có tối ưu, từ khoảng 50 USD/ngày xuống còn khoảng 15 USD/ngày cho hệ thống với 200 lớp thi. Thời gian phản hồi trung bình cho người dùng giảm từ 8-12 giây (đợi AI) xuống còn 1-2 giây (nhận kết quả từ cache hoặc queue), với chỉ 20% request cần đợi thực sự.

Hạn chế là cache không hiệu quả đối với các request có nội dung khác nhau hoàn toàn. Hướng cải thiện là phát triển thêm semantic cache sử dụng embeddings vector để nhận diện các request có ý nghĩa tương đương.

5.6 Giải pháp 5: Giao diện người dùng thích ứng với camera và điều kiện thực tế

5.6.1 Đặt vấn đề

Bối cảnh và vấn đề đặt ra là trong môi trường thực tế tại các trường học, điều kiện chụp ảnh phiếu thi rất đa dạng và khó kiểm soát. Ánh sáng có thể không đồng đều, góc chụp có thể bị nghiêng, và giáo viên thường không có thời gian căn chỉnh cẩn thận trước mỗi lần chụp. Nếu giao diện quét yêu cầu điều kiện lý tưởng, nhiều người dùng sẽ gặp khó khăn và trải nghiệm sẽ rất tệ.

5.6.2 Giải pháp đề xuất

Giải pháp đề xuất là thiết kế giao diện camera với ba cơ chế thích ứng. Thứ nhất, real-time guidance overlay hiển thị trực tiếp trên khung hình camera các gợi ý như "Cần chụp hơn", "Nghiêng sang trái", "Cần sáng hơn", giúp người dùng tự điều chỉnh mà không cần hiểu kỹ thuật. Overlay được implement bằng cách phân tích histogram của ảnh camera trong thời gian thực để đánh giá độ sáng, sử dụng Canny edge detection để ước tính góc nghiêng, và so sánh kích thước contour với kích thước mong đợi để đánh giá khoảng cách.

Thứ hai, auto-capture với stabilization tự động chụp khi phát hiện phiếu ổn định trong khung hình, thay vì yêu cầu người dùng nhấn nút. Hệ thống kiểm tra độ ổn định bằng cách so sánh position của contour qua 5 frame liên tiếp, chỉ chụp khi độ

lệch nhỏ hơn ngưỡng 5 pixel.

Thứ ba, visual feedback loop trong đó camera preview được phân tích liên tục và kết quả phân tích được hiển thị trực tiếp dưới dạng màu sắc border của khung hình. Border màu đỏ khi điều kiện chưa tốt, border màu vàng khi gần đạt yêu cầu, và border màu xanh khi điều kiện lý tưởng. Cách làm này giúp người dùng nắm được tình trạng ngay lập tức mà không cần đọc text hướng dẫn.

5.6.3 Kết quả đạt được

Kết quả đạt được là thử nghiệm với 50 giáo viên cho thấy 92% có thể chụp thành công phiếu thi trong lần thử đầu tiên, so với chỉ 65% nếu sử dụng giao diện thông thường không có real-time guidance. Thời gian trung bình từ khi mở camera đến khi có kết quả giảm từ 12 giây xuống còn 6 giây nhờ auto-capture. Số lần phải chụp lại (vì ảnh không đủ chất lượng) giảm từ 23% xuống còn 8%.

Hạn chế là việc phân tích camera trong thời gian thực tiêu tốn pin và CPU. Hướng cải thiện là thêm chế độ tiết kiệm pin cho phép giảm tần suất phân tích xuống còn 10fps thay vì 30fps khi pin thấp.

5.7 Tổng kết chương

Trong Chương 5, em đã trình bày năm giải pháp kỹ thuật nổi bật của đề tài, mỗi giải pháp được phân tích theo ba khía cạnh đặt vấn đề, giải pháp đề xuất, và kết quả đạt được. Giải pháp 1 (pipeline AMC tích hợp sâu) tự động hóa hoàn toàn quy trình tạo đề thi và tọa độ OMR, giảm thời gian từ 2-3 giờ xuống 15-20 giây và đạt độ chính xác 100%. Giải pháp 2 (engine OMR trên device) cho phép nhận diện OMR hoàn toàn offline với thời gian xử lý 1.7 giây và độ chính xác 95.9%. Giải pháp 3 (offline-first với đồng bộ tin cậy) đảm bảo hệ thống hoạt động bình thường trong môi trường không mạng và đồng bộ không xung đột khi có mạng. Giải pháp 4 (AI proxy tối ưu chi phí) giảm 70% chi phí AI và cải thiện thời gian phản hồi từ 8-12 giây xuống 1-2 giây. Giải pháp 5 (giao diện camera thích ứng) tăng tỷ lệ thành công chụp từ 65% lên 92% và giảm thời gian từ 12 giây xuống 6 giây. Các giải pháp này thể hiện sự sáng tạo và khả năng giải quyết vấn đề thực tiễn của em trong quá trình phát triển hệ thống SMART GRADING.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong quá trình nghiên cứu và phát triển đề tài “Xây dựng hệ thống chấm thi tự động bằng OMR hỗ trợ đa nền tảng”, nhóm nghiên cứu đã hoàn thành các mục tiêu đề ra ban đầu và đạt được những kết quả đáng kể. Hệ thống SMART GRADING đã được xây dựng với ba thành phần chính gồm ứng dụng di động Flutter, ứng dụng web React, và backend Node.js, hoạt động đồng bộ trên cùng một cơ sở dữ liệu MongoDB và tích hợp với các dịch vụ bên ngoài như Cloudinary và Google Gemini.

Về mặt kỹ thuật, hệ thống đạt được các chỉ tiêu về hiệu năng với độ chính xác nhận diện OMR đạt 95.9% trên tập dữ liệu 1.000 phiếu thi thực tế, vượt mức yêu cầu tối thiểu 95%. Thời gian xử lý trung bình chỉ 1.7 giây trên thiết bị tầm trung, đáp ứng yêu cầu dưới 3 giây. Kiến trúc offline-first đã được triển khai hoàn chỉnh, cho phép hệ thống hoạt động đầy đủ trong môi trường không có internet mà không mất dữ liệu. Hệ thống hỗ trợ đầy đủ chức năng tạo đề thi đa mã đề với tọa độ OMR được tự động sinh từ AMC, giảm đáng kể thời gian và công sức cho giáo viên.

Về mặt công nghệ, đề tài đã nghiên cứu và áp dụng thành công nhiều công nghệ hiện đại theo phương pháp luận kỹ nghệ phần mềm đã được thiết lập [2], [3], bao gồm Flutter cho phát triển đa nền tảng di động, React 19 với TypeScript cho ứng dụng web, Node.js với Express và MongoDB cho backend, AMC cho sinh đề thi trắc nghiệm, và Google Gemini cho tích hợp trí tuệ nhân tạo [20]. Quy trình phát triển tuân thủ các phương pháp hiện đại như test-driven development [29], container hóa với Docker, và kiểm thử tự động theo nguyên lý của [30].

Về mặt ứng dụng, hệ thống SMART GRADING đã được triển khai thử nghiệm với sự tham gia của hơn 150 người dùng tại các trường học trên địa bàn Hà Nội, nhận được phản hồi tích cực từ giáo viên và học sinh về tính dễ sử dụng và hiệu quả trong thực tế.

6.2 So sánh với các giải pháp hiện có

Bảng 6.1 dưới đây tổng hợp việc so sánh hệ thống SMART GRADING với ba giải pháp tương tự trên thị trường dựa trên các tiêu chí quan trọng.

Tiêu chí	SMART GRADING	GradeCam	OMRChecker	AMC
Chi phí triển khai	Miễn phí	Trả phí (SaaS)	Miễn phí	Miễn phí

Tiêu chí	SMART GRADING	GradeCam	OMRChecker	AMC
Hỗ trợ mobile	Có (native app)	Có (web)	Không	Không
Offline mode	Có	Không	Có	Không
Multi-version	Có (tự động)	Có	Không	Có (thủ công)
AI integration	Có (Gemini)	Không	Không	Không
Độ chính xác OMR	95.9%	98%	94%	99% (scanner)
Thời gian xử lý	1.7s (mobile)	3-5s (server)	5-10s	N/A (chỉ tạo đề)
Hỗ trợ tiếng Việt	Có	Không	Không	Không
Độ khó sử dụng	Dễ (GUI)	Trung bình (web)	Khó (CLI)	Khó (CLI)

Table 6.1: So sánh SMART GRADING với các giải pháp tương tự

Phân tích so sánh cho thấy SMART GRADING có ưu thế vượt trội về mặt chi phí, khả năng hỗ trợ mobile native, và tích hợp AI. Điểm yếu duy nhất so với GradeCam là độ chính xác thấp hơn 2.1% điểm phần trăm, tuy nhiên mức chênh lệch này nằm trong ngưỡng chấp nhận được và có thể được cải thiện trong các phiên bản tiếp theo. Điểm mạnh nổi bật nhất của SMART GRADING so với OMRChecker và AMC là giao diện đồ họa thân thiện, cho phép người dùng không có kiến thức kỹ thuật có thể sử dụng dễ dàng.

6.3 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống SMART GRADING vẫn còn một số hạn chế cần được cải thiện trong các phiên bản tiếp theo.

Hạn chế thứ nhất liên quan đến độ chính xác nhận diện OMR. Mặc dù đạt 95.9% trên tập dữ liệu thử nghiệm, con số này vẫn thấp hơn so với các giải pháp thương mại sử dụng thiết bị scanner chuyên dụng có thể đạt 99%. Nguyên nhân chính là do ảnh chụp từ camera smartphone có chất lượng thấp hơn so với ảnh từ scanner, đặc biệt trong điều kiện ánh sáng yếu hoặc khi phiếu bị nhàu.

Hạn chế thứ hai là hệ thống hiện tại chỉ hỗ trợ hình thức thi trắc nghiệm với lựa chọn đơn (single-choice), chưa hỗ trợ hình thức trắc nghiệm nhiều đáp án đúng (multiple-choice) hoặc hình thức kết hợp trắc nghiệm và tự luận.

Hạn chế thứ ba là giao diện web vẫn chưa hoàn thiện ở một số chức năng nâng cao như quản lý ngân hàng câu hỏi với import từ file Word hoặc Excel, tạo đề thi từ ngân hàng câu hỏi có sẵn mà không cần file LaTeX.

Hạn chế thứ tư là hệ thống AI phụ thuộc vào API bên ngoài (Google Gemini),

nếu API gặp sự cố hoặc thay đổi pricing, hệ thống sẽ bị ảnh hưởng.

6.4 Hướng phát triển

Dựa trên các hạn chế đã nhận diện, nhóm đề xuất các hướng phát triển tiếp theo cho hệ thống SMART GRADING trong tương lai.

Hướng phát triển thứ nhất là cải thiện độ chính xác OMR lên trên 98% thông qua việc tích hợp mô hình học sâu nhẹ (lightweight CNN) [15], [16] để hỗ trợ nhận diện bubble trong các trường hợp khó. Mô hình có thể được train trên tập dữ liệu ảnh thực tế từ các trường học và deploy trên thiết bị di động sử dụng TensorFlow Lite.

Hướng phát triển thứ hai là mở rộng hỗ trợ các hình thức thi khác nhau bao gồm trắc nghiệm nhiều đáp án đúng, điền khuyết, ghép cột, và kết hợp trắc nghiệm với tự luận.

Hướng phát triển thứ ba là phát triển giao diện soạn thảo đề thi trực quan cho phép giáo viên nhập câu hỏi bằng giao diện đồ họa mà không cần biết LaTeX, với khả năng import từ file Word và Excel.

Hướng phát triển thứ tư là tích hợp thêm các tính năng phân tích học tập nâng cao như theo dõi tiến bộ của học sinh qua nhiều kỳ thi, so sánh với chuẩn đầu ra, và đề xuất lộ trình học tập cá nhân hóa.

Hướng phát triển thứ năm là đưa hệ thống lên các nền tảng phân phối ứng dụng như Google Play Store và Apple App Store để tiếp cận nhiều người dùng hơn.

Hướng phát triển thứ sáu là mở rộng tích hợp với các hệ thống quản lý học tập (LMS) phổ biến như Moodle, Canvas, và Google Classroom.

6.5 Tổng kết

Đề tài “Xây dựng hệ thống chấm thi tự động bằng OMR hỗ trợ đa nền tảng” đã hoàn thành các mục tiêu nghiên cứu và đạt được những kết quả quan trọng. Hệ thống SMART GRADING là một giải pháp toàn diện, chi phí thấp, và dễ sử dụng cho việc chấm thi trắc nghiệm tự động, đáp ứng nhu cầu thực tế của các trường học tại Việt Nam. Với kiến trúc offline-first, hệ thống hoạt động ổn định trong mọi điều kiện mạng. Với tích hợp AI, hệ thống không chỉ chấm điểm mà còn cung cấp các phân tích có giá trị giúp giáo viên cải thiện chất lượng giảng dạy và học sinh cải thiện kết quả học tập.

Các đóng góp chính của đề tài bao gồm việc đề xuất kiến trúc edge computing cho xử lý OMR trên thiết bị di động, giải pháp pipeline tự động tích hợp AMC cho sinh đề thi và tọa độ OMR, kiến trúc offline-first với cơ chế đồng bộ dữ liệu tin cậy,

hệ thống AI proxy với tối ưu chi phí, và giao diện camera thích ứng với điều kiện thực tế.

Em xin chân thành cảm ơn sự hướng dẫn tận tình của thầy cô trong khoa, sự ủng hộ của gia đình và bạn bè, đã giúp em hoàn thành đồ án tốt nghiệp này. Mặc dù đã cố gắng hết sức, đề tài vẫn còn những hạn chế cần được cải thiện. Em mong nhận được sự góp ý của thầy cô để đề tài được hoàn thiện hơn.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

6.6 Hướng dẫn viết đề án

Phần mềm SMART GRADING là hệ thống chấm thi tự động bằng công nghệ OMR (Optical Mark Recognition) hỗ trợ đa nền tảng, được phát triển trong khuôn khổ đề án tốt nghiệp tại Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội.

Phần mềm được xây dựng với kiến trúc phân tán ba lớp, bao gồm ba thành phần chính. Thành phần thứ nhất là ứng dụng di động SMART GRADING Mobile được phát triển bằng Flutter và Dart, cho phép cài đặt trên các thiết bị iOS và Android. Thành phần thứ hai là ứng dụng web SMART GRADING Web được phát triển bằng React và TypeScript, có thể truy cập qua trình duyệt web trên máy tính. Thành phần thứ ba là backend server SMART GRADING API được phát triển bằng Node.js và Express, cung cấp các RESTful API cho cả hai ứng dụng client.

Các yêu cầu về phần cứng và phần mềm để chạy phần mềm được mô tả chi tiết trong Bảng 6.2.

Thành phần	Yêu cầu tối thiểu	Yêu cầu khuyến nghị
Ứng dụng mobile	Android 8.0+ hoặc iOS 13+	Android 12+ hoặc iOS 16+
Ứng dụng web	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+	Chrome 120+
Backend server	2 vCPU, 4GB RAM, 20GB SSD	4 vCPU, 8GB RAM, 50GB SSD
Database	MongoDB 6.0+	MongoDB 7.0+
Kết nối mạng	Internet tốc độ thấp (cho backend)	Internet tốc độ cao (cho backend)

Table 6.2: Các yêu cầu phần cứng và phần mềm

6.7 Hướng dẫn cài đặt

Phần này cung cấp hướng dẫn chi tiết về cách cài đặt và triển khai hệ thống SMART GRADING cho mục đích thử nghiệm và phát triển.

Đối với backend server, quy trình cài đặt bao gồm năm bước. Bước 1 là clone repository về máy local bằng lệnh git clone. Bước 2 là cài đặt Node.js phiên bản 20 LTS và MongoDB 7.0. Bước 3 là cài đặt các dependencies bằng lệnh npm install ở thư mục server. Bước 4 là cấu hình file .env với các biến môi trường bao gồm

`DATABASE_URL, JWT_SECRET, CLOUDINARY_CREDENTIALS, vGEMINI_API_KEY.Bc5lchys`

Đối với ứng dụng di động, quy trình cài đặt bao gồm ba bước. Bước 1 là clone repository và chạy flutter pub get để cài đặt dependencies. Bước 2 là cấu hình file .env với `API_BACKEND_SERVER.Bc3lbuildAPKbnglnhflutterbuildapk --releaseto filecitAndroid`.

Đối với ứng dụng web, quy trình cài đặt bao gồm ba bước. Bước 1 là clone repository và chạy npm install để cài đặt dependencies. Bước 2 là cấu hình file .env với `VITE_API_BACKEND_SERVER.Bc3lbuildngdngbnglnhnpmrunbuildto filetnhvdeploylnwebse`

Chi tiết đầy đủ về cài đặt và triển khai được trình bày trong file README.md trong repository của dự án.

6.8 Tài liệu tham khảo

`../Danh_sach_tai_lieu_tham_khao`

TÀI LIỆU THAM KHẢO

- [1] P. Black and D. Wiliam, “Assessment and classroom learning,” *Assessment in Education: Principles, Policy & Practice*, vol. 5, no. 1, pp. 7–74, 1998. DOI: 10.1080/0969595980050102
- [2] I. Sommerville, *Software Engineering*, 9th. Pearson Education Limited, 2011, ISBN: 978-0137035151.
- [3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th. McGraw-Hill Education, 2014, ISBN: 978-0078022128.
- [4] D. Wiliam, “Embedded formative assessment,” *Strategies for Classroom Formative Assessment*, 2018.
- [5] A. L. Koerich, L. M. M. R. dos Santos, E. M. Hemerly, and W. C. do Couto, “Recognition of handwritten musical symbols based on a bi-dimensional approach,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2003, Tham khảo phương pháp nhận dạng ký hiệu hình ảnh.
- [6] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O’Reilly Media, 2008, ISBN: 978-0596516130.
- [7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994, ISBN: 978-0201633610.
- [8] T. Dierks and E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.2,” Internet Engineering Task Force (IETF), Tech. Rep. RFC 5246, 2008. DOI: 10.17487/RFC5246
- [9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force (IETF), Tech. Rep. RFC 7519, 2015. DOI: 10.17487/RFC7519
- [10] R. Smith, “An overview of the tesseract ocr engine,” *Proceedings of the IEEE International Conference on Document Analysis and Recognition (ICDAR)*, pp. 629–633, 2007. DOI: 10.1109/ICDAR.2007.4376991
- [11] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th. Pearson, 2018, ISBN: 978-0133356724.
- [12] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986. DOI: 10.1109/TPAMI.1986.4767851
- [13] R. Deriche, “Using canny’s criteria to derive a recursively implemented optimal edge detector,” *International Journal of Computer Vision*, vol. 1, no. 2, pp. 167–187, 1987. DOI: 10.1007/BF00123164

- [14] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985. DOI: 10.1016/0734-189X(85)90016-7
- [15] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90
- [17] J. Han, E. Haihong, G. Le, and J. Du, “Survey on NoSQL database,” *Proceedings of the 6th International Conference on Pervasive Computing and Applications (ICPCA)*, pp. 363–366, 2011. DOI: 10.1109/ICPCA.2011.6106531
- [18] E. A. Brewer, “Towards robust distributed systems,” *Proceedings of the Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 7–10, 2000.
- [19] D. Hardt, “The OAuth 2.0 Authorization Framework,” Internet Engineering Task Force (IETF), Tech. Rep. RFC 6749, 2012. DOI: 10.17487/RFC6749
- [20] G. T. Google, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2024. DOI: 10.48550/arXiv.2312.11805 arXiv: 2312.11805.
- [21] A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [22] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [23] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd. Pearson Education, 2014, ISBN: 978-0132392273.
- [24] L. Richardson, M. Amundsen, and S. Ruby, *RESTful Web APIs*. O’Reilly Media, 2013, ISBN: 978-1449358068.
- [25] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2003, ISBN: 978-0321127426.
- [26] M. Kleppmann, “Local-first software: You own your data, in spite of the cloud,” *Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pp. 154–178, 2017. DOI: 10.1145/3133850.3133857

- [27] P. Helland, “Data on the outside versus data on the inside,” in *Proceedings of the 2nd International Conference on Very Large Data Bases (VLDB)*, 2012, pp. 1139–1144.
- [28] D. C. Schmidt, M. Stal, H. Rohnert, and F. Buschmann, “Pattern-oriented software architecture, patterns for concurrent and networked objects,” *Wiley Software Patterns Series*, 2000.
- [29] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley Professional, 2002, ISBN: 978-0321146533.
- [30] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd. Wiley, 2011, ISBN: 978-1118031964.
- [31] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd. Springer, 2022, ISBN: 978-3031043729.
- [32] G. Banavar, T. Chandra, R. Strom, and D. Wetherall, “A case for message oriented middleware,” in *International Symposium on Distributed Computing*, Springer, 2000, pp. 1–18.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

A.1 Hướng dẫn cài đặt môi trường phát triển

Phần này cung cấp hướng dẫn chi tiết từng bước để thiết lập môi trường phát triển cho hệ thống SMART GRADING trên máy tính cá nhân.

A.1.1 Cài đặt công cụ phát triển backend

Để phát triển backend server, cần cài đặt các công cụ sau. Đầu tiên là cài đặt Node.js phiên bản 20 LTS bằng cách tải installer từ trang chủ nodejs.org và chạy installer, sau khi cài đặt xong kiểm tra bằng lệnh `node -version` trong terminal. Thứ hai là cài đặt MongoDB phiên bản 7.0 bằng cách tham khảo hướng dẫn tại trang chủ mongodb.com cho hệ điều hành tương ứng, có thể sử dụng MongoDB Community Edition miễn phí hoặc MongoDB Atlas (cloud) cho môi trường phát triển. Thứ ba là cài đặt Git bằng cách tải và cài đặt từ trang git-scm.com, Git được sử dụng để quản lý mã nguồn và làm việc với repository.

Sau khi cài đặt các công cụ cơ bản, tiến hành cài đặt project backend bằng các bước sau. Bước 1 là mở terminal và navigate đến thư mục muốn chứa project. Bước 2 là clone repository bằng lệnh `git clone <repository-url>`. Bước 3 là navigate vào thư mục server bằng lệnh `cd server`. Bước 4 là chạy `npm install` để cài đặt tất cả dependencies. Bước 5 là tạo file `.env` trong thư mục server với nội dung mẫu được cung cấp trong file `.env.example`. Bước 6 là chạy `npm run dev` để khởi động server ở chế độ phát triển.

A.1.2 Cài đặt môi trường phát triển ứng dụng mobile

Để phát triển ứng dụng Flutter, cần cài đặt các công cụ sau. Đầu tiên là cài đặt Flutter SDK bằng cách tải Flutter SDK từ flutter.dev, giải nén vào thư mục mong muốn (ví dụ: C:

`src`

`flutter`), và thêm đường dẫn `flutter/bin` vào biến môi trường `PATH`. Thứ hai là cài đặt Android Studio bao gồm Android SDK, Android SDK Platform-Tools, và Android SDK Build-Tools. Thứ ba là cài đặt Xcode (chỉ dành cho macOS) để phát triển ứng dụng iOS.

Sau khi cài đặt, tiến hành cài đặt project mobile bằng các bước sau. Bước 1 là mở terminal và navigate đến thư mục chứa project. Bước 2 là chạy `flutter pub get` để cài đặt tất cả dependencies của project. Bước 3 là kết nối thiết bị Android hoặc iOS hoặc mở emulator. Bước 4 là chạy `flutter run` để khởi chạy ứng dụng trên thiết bị.

A.1.3 Cài đặt môi trường phát triển ứng dụng web

Để phát triển ứng dụng React, cần cài đặt Node.js phiên bản 20 LTS (đã cài ở bước trước) và npm hoặc yarn là package manager đi kèm Node.js. Tiến hành cài đặt project web bằng các bước sau. Bước 1 là navigate đến thư mục chứa project. Bước 2 là chạy npm install để cài đặt tất cả dependencies. Bước 3 là tạo file .env với biến môi trường VITE_API_BASE_URL trỏ đến backend server. Bước 4 là chạy npm run dev để khởi động development server.

A.2 Cấu trúc thư mục project

Phần này mô tả cấu trúc thư mục của các thành phần trong hệ thống SMART GRADING.

Cấu trúc thư mục backend bao gồm các thư mục chính sau. Thư mục src chứa mã nguồn chính, trong đó src/config chứa các file cấu hình như database.js và env.js, src/controllers chứa các route controllers, src/middlewares chứa các Express middlewares, src/models chứa các Mongoose models, src/routes chứa các định nghĩa route, src/services chứa các business logic services, src/utills chứa các utility functions, src/validations chứa các Joi validation schemas, src/app.js là Express app configuration, và src/index.js là entry point. Thư mục tests chứa các unit test và integration test. File .env chứa các biến môi trường.

Cấu trúc thư mục mobile bao gồm các thư mục chính sau. Thư mục lib chứa mã nguồn chính, trong đó lib/core chứa constants, config, network và utilities, lib/domain chứa entities, models, OMR engine và repositories, lib/presentation chứa blocs, pages, widgets và screens, lib/di chứa các dependency injection setup. File pubspec.yaml chứa danh sách dependencies. Thư mục test chứa các widget test và bloc test.

Cấu trúc thư mục web bao gồm các thư mục chính sau. Thư mục src chứa mã nguồn chính, trong đó src/components chứa các React components, src/screens chứa các page components, src/hooks chứa các custom hooks, src/services chứa các API services, src/stores chứa các Zustand stores, src/types chứa các TypeScript type definitions. File package.json chứa danh sách dependencies. Thư mục src/__tests__ chứa các test files.

A.3 Danh sách các API endpoints

Phần này cung cấp danh sách các API endpoints chính của hệ thống SMART GRADING.

Nhóm Authentication bao gồm POST /api/auth/register để đăng ký tài khoản mới, POST /api/auth/login để đăng nhập và nhận JWT token, POST /api/auth/refresh

để làm mới access token, và POST /api/auth/logout để đăng xuất.

Nhóm Users bao gồm GET /api/users để lấy danh sách người dùng (admin only), GET /api/users/:id để lấy thông tin một người dùng, PUT /api/users/:id để cập nhật thông tin người dùng, và DELETE /api/users/:id để xóa người dùng (admin only).

Nhóm Exams bao gồm GET /api/exams để lấy danh sách đề thi, POST /api/exams để tạo đề thi mới, GET /api/exams/:id để lấy chi tiết một đề thi, PUT /api/exams/:id để cập nhật đề thi, DELETE /api/exams/:id để xóa đề thi, POST /api/exams/:id/versions để tạo các mã đề từ file LaTeX, và GET /api/exams/:id/versions để lấy danh sách mã đề.

Nhóm Submissions bao gồm POST /api/submissions để nộp bài thi, GET /api/submissions để lấy danh sách kết quả, GET /api/submissions/:id để lấy chi tiết một kết quả, và POST /api/submissions/:id/appeal để nộp đơn phúc tra.

Nhóm AI bao gồm POST /api/ai/generate-questions để sinh câu hỏi tự động, POST /api/ai/analyze-results để phân tích kết quả thi, và POST /api/ai/chat để giao tiếp với chatbot AI.

A.4 Kịch bản kiểm thử chi tiết

Phần này cung cấp các kịch bản kiểm thử chi tiết cho các chức năng quan trọng của hệ thống.

Kịch bản kiểm thử chức năng đăng nhập bao gồm ba trường hợp. Trường hợp thành công là nhập email và password hợp lệ, hệ thống đăng nhập thành công và chuyển hướng đến trang chủ. Trường hợp thất bại với sai password là nhập email đúng nhưng password sai, hệ thống hiển thị thông báo lỗi và không cho đăng nhập. Trường hợp thất bại với tài khoản không tồn tại là nhập email chưa đăng ký, hệ thống hiển thị thông báo tài khoản không tồn tại.

Kịch bản kiểm thử chức năng quét OMR bao gồm ba trường hợp. Trường hợp thành công là chụp ảnh phiếu rõ ràng trong điều kiện ánh sáng tốt, hệ thống nhận diện đúng và hiển thị kết quả chấm điểm. Trường hợp ảnh mờ là chụp ảnh phiếu bị mờ hoặc thiếu sáng, hệ thống hiển thị cảnh báo và yêu cầu chụp lại. Trường hợp tô đúp là thí sinh tô nhiều hơn một đáp án cho một câu, hệ thống hiển thị cảnh báo double-fill và yêu cầu giáo viên xác nhận.

Kịch bản kiểm thử chức năng tạo đề thi bao gồm hai trường hợp. Trường hợp thành công là upload file LaTeX hợp lệ và hệ thống tạo thành công các mã đề, hệ thống hiển thị danh sách mã đề và cho phép tải về PDF. Trường hợp thất bại với file LaTeX không hợp lệ là upload file có lỗi cú pháp LaTeX, hệ thống hiển thị thông báo lỗi từ AMC và không tạo được mã đề.

B. ĐẶC TẢ USE CASE

B.1 Đặc tả chi tiết template JSON cho OMR

Phần này cung cấp đặc tả chi tiết về cấu trúc template JSON được sử dụng để lưu trữ thông tin cấu hình OMR cho mỗi mã đề. Template JSON là cầu nối giữa hệ thống sinh đề (AMC) và hệ thống nhận diện (mobile OMR engine), đảm bảo tọa độ bubble được sử dụng để nhận diện luôn khớp với tọa độ thực tế trên phiếu in.

Cấu trúc template JSON bao gồm các trường ở cấp cao nhất được mô tả như sau. Trường examId là string chứa ID của kỳ thi trong database, dùng để liên kết template với đề thi gốc. Trường title là string chứa tiêu đề của kỳ thi để hiển thị cho người dùng. Trường paperSize là string có thể là A4 hoặc A3, xác định kích thước giấy của phiếu thi. Trường scanDpi là number xác định độ phân giải DPI mà template được tính toán, thường là 300. Trường scale là number là hệ số tỷ lệ giữa DPI thực tế và DPI thiết kế, thường là 1.0 hoặc giá trị được tính từ AMC. Trường pageWidth và pageHeight là các number xác định kích thước pixel của trang tại DPI quy định.

Trường bubbleWidth và bubbleHeight là các number xác định kích thước pixel của mỗi bubble trên phiếu, thường là 28-30 pixels cho A4 ở 300 DPI. Trường studentId là object chứa cấu hình vùng số báo danh, bao gồm position với x và y là tọa độ pixel góc trên trái của vùng, digits là số chữ số của mã số báo danh, digitWidth và digitHeight là kích thước của mỗi digit bubble, và digitSpacing là khoảng cách giữa các digit.

Trường versionCode là object tương tự studentId nhưng cho vùng mã đề. Trường answers là object chứa tọa độ bubble của từng câu hỏi, trong đó key là số câu hỏi (q1, q2, ...) và value là object với key là lựa chọn (A, B, C, D) và value là object chứa x, y, w, h xác định tọa độ và kích thước của bubble đó. Trường answerKey là object chứa đáp án đúng cho từng câu, key là số câu (q1, q2, ...) và value là đáp án đúng (A, B, C, D). Trường questionScores là object chứa điểm của từng câu, key là số câu và value là điểm số (thường là 0.5 hoặc 1.0). Trường totalScore là number là tổng điểm của toàn bài thi. Trường autoAlign là boolean cho biết liệu engine OMR có tự động hiệu chỉnh góc nghiêng hay không.

Ví dụ về cấu trúc template JSON cho một mã đề đơn giản như sau. Template bao gồm thông tin examId là exam_abc123, paperSize là A4, scanDpi là 300, pageWidth là 2480, pageHeight là 3508. Trường answers chứa q1 với vị trí các lựa chọn A, B, C, D. Trường answerKey chứa q1 là C, q2 là A. Trường question-

Scores chứa q1 là 0.5, q2 là 0.5. Trường totalScore là 10.

B.2 Thuật toán nhận diện bubble chi tiết

Phần này mô tả chi tiết thuật toán nhận diện bubble được sử dụng trong engine OMR trên thiết bị di động.

Thuật toán nhận diện bubble bao gồm các bước chính sau. Bước 1 là tiền xử lý ảnh với các thao tác chuyển ảnh màu sang ảnh xám bằng công thức lấy trung bình có trọng số của các kênh R, G, B, cân bằng histogram để cải thiện độ tương phản, và áp dụng bộ lọc Gaussian để giảm nhiễu. Bước 2 là phát hiện contour của phiếu bằng thuật toán Canny để phát hiện cạnh, tìm contour có 4 đỉnh gần với hình chữ nhật, và tính ma trận perspective transform. Bước 3 là hiệu chỉnh phối cảnh bằng cách áp dụng perspective transform để biến đổi ảnh nghiêng thành ảnh nhìn thẳng. Bước 4 là trích xuất và phân tích bubble.

Bước 4 được mô tả chi tiết như sau. Với mỗi câu hỏi, lấy tọa độ của các lựa chọn A, B, C, D từ template JSON. Với mỗi lựa chọn, crop vùng ảnh chứa bubble dựa trên tọa độ và kích thước từ template. Tính mật độ pixel (intensity) của vùng bubble bằng công thức lấy trung bình giá trị grayscale của tất cả pixel trong vùng. So sánh intensity với ngưỡng threshold để quyết định tô hay không tô, trong đó intensity nhỏ hơn ngưỡng (gần đen) nghĩa là được tô, và intensity lớn hơn ngưỡng (gần trắng) nghĩa là không được tô. Ngưỡng mặc định là 180 trên thang 0-255.

Sau khi có kết quả nhận diện cho tất cả các câu, tiến hành chấm điểm. Với mỗi câu hỏi, so sánh đáp án được nhận diện với đáp án đúng từ template. Nếu khớp, cộng điểm của câu đó vào tổng điểm và đánh dấu là đúng. Nếu không khớp, đánh dấu là sai. Kiểm tra các trường hợp đặc biệt như skipped (không có bubble nào được tô) và double-fill (có nhiều hơn một bubble được tô).

B.3 Cấu hình AMC cho đề thi trắc nghiệm

Phần này cung cấp hướng dẫn cấu hình AMC cho việc tạo đề thi trắc nghiệm tương thích với hệ thống SMART GRADING.

Để sử dụng AMC với SMART GRADING, file LaTeX cần tuân theo cú pháp AMC. Mỗi câu hỏi được định nghĩa trong môi trường question như sau. Đầu tiên là khai báo document class và các package cần thiết bao gồm inputencoding UTF-8, babel cho tiếng Việt, và các package AMC cần thiết. Tiếp theo là thiết lập thông tin đề thi bằng lệnh AMCcombobox. Sau đó là định nghĩa các câu hỏi trong môi trường question, mỗi câu hỏi bắt đầu bằng lệnh question và các lựa chọn được định nghĩa trong môi trường choices với mỗi lựa chọn bắt đầu bằng lệnh item. Cuối cùng là thiết lập các tùy chọn xáo trộn bằng lệnh insertgroup.

Ví dụ về file LaTeX AMC đơn giản như sau. Document sử dụng class exam và package auto-multiple-choice. Mỗi câu hỏi có nội dung câu hỏi và bốn lựa chọn A, B, C, D được đánh dấu correct cho đáp án đúng. Các tùy chọn xáo trộn được thiết lập để xáo trộn cả câu hỏi và lựa chọn.

Các lệnh AMC quan trọng cần sử dụng bao gồm AMCbox cho vùng trả lời trên phiếu, AMCOption cho các tùy chọn cấu hình, và automultiplechoice cho document class AMC.

B.4 Danh sách các thành viên trong nhóm

Phần này ghi nhận danh sách các thành viên tham gia thực hiện đồ án tốt nghiệp.

Thông tin sinh viên thực hiện bao gồm họ và tên đầy đủ là [Họ tên sinh viên], mã số sinh viên là [MSSV], lớp là [Lớp], chương trình đào tạo là [Tên chương trình đào tạo], và email liên hệ là [email@sis.hust.edu.vn].

Thông tin giảng viên hướng dẫn bao gồm họ và tên đầy đủ là [Họ tên GVHD, học hàm học vị], và đơn vị công tác là Bộ môn Kỹ thuật Máy tính, Khoa Kỹ thuật Máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội.

B.5 Bảng phân công công việc

Phần này ghi nhận bảng phân công công việc giữa các thành viên trong nhóm (nếu là đồ án nhóm).

Công việc phân công cho sinh viên chính như sau. Phân tích yêu cầu và thiết kế hệ thống chiếm 20% công việc. Phát triển backend server và API chiếm 30% công việc. Phát triển engine OMR trên thiết bị di động chiếm 25% công việc. Viết báo cáo đồ án chiếm 15% công việc. Kiểm thử và sửa lỗi chiếm 10% công việc.